

The ceBWT Index: An Index for Circular Cartesian Tree Matching on Multiple Texts*

Eric M. Osterkamp ✉ 

University of Münster, osterkamp@uni-muenster.de, Germany

Dominik Köppl ✉ 

University of Yamanashi, dkppl@yamanashi.ac.jp, Japan

Abstract

Cartesian tree matching is a form of generalized pattern matching where a substring of the text matches with the pattern if they share the same Cartesian tree. This form of matching finds application for time series of stock prices and can be of interest for melody matching between musical scores. For the indexing problem, the state-of-the-art data structure is a Burrows–Wheeler transform based solution due to [Kim and Cho, CPM’21], which uses nearly succinct space and can count the number of substrings that Cartesian tree match with a pattern in time linear in the pattern length. The authors address the construction of their data structure with a straightforward solution that, however, requires pointer-based data structures, resulting in $O(n \lg n)$ bits of space, where n is the text length [Kim and Cho, CPM’21, Section A.4]. We address this bottleneck by a construction that requires $O(n \lg \sigma)$ bits of space and has a time complexity of $O(n \frac{\lg \sigma \lg n}{\lg \lg n})$, where σ is alphabet size. Additionally, we can extend Kim and Cho’s index for indexing multiple circular texts in the spirit of the extended Burrows–Wheeler transform. We show that the extended index maintains the same complexities and present a dynamic variant, where we pay a logarithmic slowdown and need space linear in the input texts in bits for the extra functionality that we can incrementally add and remove texts. As an application, we give the computation of circular matching statistics. Our extended setting is of interest for finding repetitive motifs common in the aforementioned applications, independent of offsets and scaling.

2012 ACM Subject Classification Theory of computation

Keywords and phrases Cartesian-tree matching, extended Burrows–Wheeler transform, construction algorithm, generalized pattern matching

Funding *Dominik Köppl*: JSPS KAKENHI Grant Numbers JP23H04378 and JP25K21150, and Yamanashi Wakate Grant Number 2291

Acknowledgements We sincerely thank the anonymous reviewers for their constructive comments.

1 Introduction

String matching is ubiquitous in computer science, and its variations are custom-made to solve a wide variety of problems. We here focus on a special kind of variation called *substring consistent equivalence relation* (SCER) [24]. Two strings X and Y are said to SCER-match if they have the same length and all substrings of equal length starting at the same text position SCER-match, i.e., $X[i..j]$ SCER-matches with $Y[i..j]$ for all $1 \leq i \leq j \leq |X| = |Y|$. A specific instance of SCER-matching is *order-preserving matching* [22, 19], which has been studied for the analysis of numerical time series. The aim of order-preserving matching is to match two strings if the relative order of the symbols in both strings is the same. Order-preserving matching therefore can find matches independently of value offsets and scaling.

* This article is an extension of a contribution [31] to the 36th Annual Symposium on Combinatorial Pattern Matching (CPM 2025).

Since order-preserving matching takes the global order of the symbols in a string into account, it may be too strict in applications that primarily consider the local ordering of ranges partitioned by the peaks. In fact, time series of stock prices are such a case, where a common pattern called the *head-and-shoulder* [11] involves one absolute peak (head) and neighboring local peaks (shoulders), where each of these neighboring peaks can be treated individually to match similar head-and-shoulder patterns with slightly changed local peaks. For such a reason, Park et al. [32] proposed *Cartesian tree matching*. Cartesian tree matching relaxes the notion of order-preserving matching in the sense that an order-preserving match is always a Cartesian tree match, but not necessarily the other way around. For instance, the two strings of digits 1637452 and 2438372 fit into the head-and-shoulder pattern, do not order-preserving match, but Cartesian tree match. For that, Cartesian tree matching compares the Cartesian trees of two strings to decide whether they match. The *Cartesian tree* [34] is a binary tree built upon an array of numbers. If all numbers are distinct, it is the min-heap whose in-order traversal retrieves the original array. Since its inception, Cartesian tree matching and variations thereof have attracted interest from researchers, whose studies include multiple pattern matching [32, 33], approximate matching [1, 21], substring matching [6], subsequence matching [29], indeterminate matching [13], cover matching [18], and palindromic matching [12].

In addition to stock prices, applications of generalized pattern matching can also be found in melody matching between musical scores. Pattern matching music scores such as differences, directions, or magnitudes have been studied [15]. However, the detection of repetitions in a piece of music has also been considered of importance [10]. The question therefore is whether we can find repetitive substrings that Cartesian tree match with a repetition of a set of melodic motifs (i.e., input texts). For that to be efficient, we want to index these motifs.

An index for Cartesian tree matching of a text string T is a data structure built upon T that can report the number of substrings of T that Cartesian tree match a given pattern. Given T has n symbols drawn from an integer alphabet of size σ , Park et al. [32] and Nishimoto et al. [27] proposed indexes for Cartesian tree matching of T that both occupy $O(n \lg n)$ bits of space and are constructed with $O(n \lg n)$ additional bits of space. Park et al. [32, Section 5.1] achieve $O(m \lg \sigma)$ time and Nishimoto et al. [27, Section 5.1] $O(m(\sigma + \lg m) + occ)$ time for answering count queries, where occ is the number of occurrences of the pattern of length m . Adapting Ferragina and Mantaci's FM-index for exact string matching [7], Kim and Cho [20] proposed an index that occupies $3n + o(n)$ bits of space, and answers a count query in $O(m)$ time for a pattern of length m . For construction, they proposed a straight-forward solution that takes as input the Cartesian suffix tree of Park et al. [32], which, however, requires $O(n \lg n)$ additional bits of space.

We address two goals in this paper. The first is a construction algorithm for Kim and Cho's index that takes $O(n \lg \sigma)$ bits of working space, which is compact for constant alphabet sizes. While we consider an index supporting access to a character of the length- n text T as compact if its space is $O(n \lg \sigma)$ bits, we here restrict queries to Cartesian-tree pattern matching, and a Cartesian tree storing n nodes can be represented in $2n$ bits, cf. [32, Section 3.5]. Therefore, a compact index for Cartesian-tree pattern matching takes $O(n)$ bits, a bound we reach for constant alphabets. Second, while all aforementioned indexes can partially address the problem by indexing multiple texts, it is hard to detect whether the pattern is a repetition of one of the input texts, which is of interest in case of indexing melodic motifs that can repeat. Here, our goal is an index that can find such matches, even if they start with different offsets of the same repetition. In concrete words, our aim is to

i	1	2	3	4	5	6	7	8
$V[i]$	7	14	5	1	11	27	11	7

■ **Figure 1** Example integer string V to illustrate queries. Here, $\text{rank}_{11}(V, 5) = 1$, $\text{rnkcnt}_V(2, 7, 7, 14) = 3$, $\text{select}_{11}(V, 2) = 7$, $\text{RNV}_V(1, 5, 8) = 11$, and $\text{MI}_V(5, 6) = [4..8]$.

index multiple texts for Cartesian pattern matching. The search space for a pattern are the texts that are considered to be infinite concatenations with themselves.

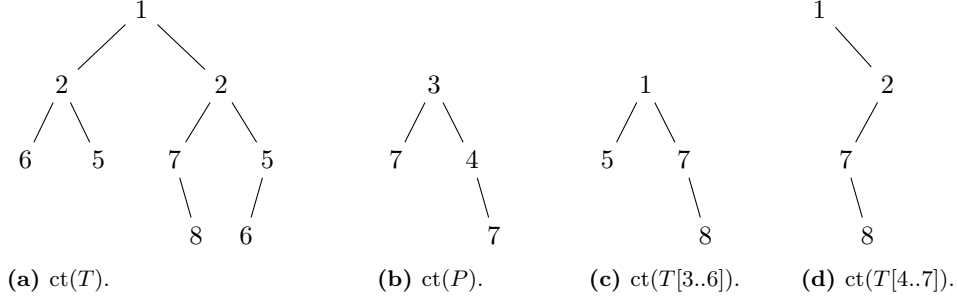
In this article, we propose an extension of the index of Kim and Cho [20] for Cartesian tree matching with techniques of the extended Burrows–Wheeler transform [23], and call the resulting data structure the *ceBWT index* (Section 3). The ceBWT index is an extension in the sense that it supports indexing multiple input texts for Cartesian tree matching circularly. We show that we can compute the ceBWT index in $O(n \frac{\lg \sigma \lg n}{\lg \lg n})$ time and $O(n \lg \sigma)$ bits of space, where n is the total length of all texts to index (Section 4). Our construction allows to build the ceBWT index incrementally, i.e., we support the incremental addition of a new string to the set of texts we index. We can also compute the original index of Kim and Cho within the same complexities. Our ideas stem from Hashimoto and colleagues’ [14] and Iseri and colleagues’ [17] construction algorithms of indexes for parameterized matching, which we recently extended for multiple circular texts [30]. During construction, the ceBWT index supports the backward search and count queries for pattern strings in $O(m \frac{\lg \sigma \lg n}{\lg \lg n})$ time, where m is the pattern length. Compared to the conference paper [31], we give extended examples (e.g., Section 3.4), present a static $3n + o(n)$ -bit index (Section 5), and give circular matching statistics as an application (Section 6).

2 Preliminaries

Let $\lg = \log_2$ denote the logarithm to base two. We assume a random access model with word size $\Omega(\lg n)$, where n denotes the input size. An interval $\{i, i+1, \dots, j-1, j\}$ of integers is denoted by $[i..j]$, where $[i..j] = \emptyset$ if $j < i$.

Strings. Let Σ denote an *alphabet*. We call elements of Σ *symbols*, a sequence of symbols from Σ a *string over Σ* , and denote the set of strings over Σ by Σ^* . Let $U, V, W \in \Sigma^*$. The *concatenation* of U and V is denoted by $U \cdot V$ or UV . If $X = UVW$, then U is called a *prefix*, W a *suffix*, and U, V, W *substrings* of X . The *length* $|U|$ of U is the number of symbols in U , $\text{lcp}(U, V)$ reports the length of the longest common prefix of U and V , ε denotes the unique string of length 0, and we define $\Sigma^+ = \Sigma^* - \{\varepsilon\}$. We write U^k if we concatenate k instances of U for a non-negative integer k , and U^ω for the string obtained by infinitely concatenating U . We call U *primitive* if $U = X^k$ implies $U = X$ and $k = 1$. It is known that for every $U \in \Sigma^+$ there exists a unique primitive $X \in \Sigma^+$ and a unique integer k such that $U = X^k$, denoted by $\text{root}(U)$ and $\text{exp}(U)$, respectively. Let $X \in \Sigma^+$ and $i, j \in [1..|X|]$. Then $X[i]$ denotes the i -th symbol in X , and $X[i..j] = X[i] \cdots X[j]$, where $X[i..j] = \varepsilon$ if $j < i$, $X[..i] = X[1..i]$, and $X[i..] = X[i..|X|]$. For notational convenience, $X[..k] = \varepsilon$ if $k \leq 0$, and $X[k..] = \varepsilon$ if $k \geq |X| + 1$. We call $1 \leq p \leq |X|$ satisfying $X[q] = X[q+p]$ for every $q \in [1..|X| - p]$ a *period* of X . Let $\text{Rot}(X, 0) = X$ and $\text{Rot}(X, k+1) = \text{Rot}(X, k)[2..] \cdot \text{Rot}(X, k)[1]$ for each non-negative integer k , i.e., $\text{Rot}(X, k)$ denotes the k -th *left rotation* of X . The left rotations of X for $k \in [0..|X| - 1]$ are the *conjugates* of X . We write $U < V$ if and only if (a) $U = V[..|U|]$ and $|U| < |V|$ or (b) $U[\text{lcp}(U, V) + 1] < V[\text{lcp}(U, V) + 1]$.

Let $V \in \Sigma^+$ be a (dynamic) string, $c \leq d \in \Sigma$, $i \in [1..|V| + 1]$, $j \in [0..|V|]$ and $k \in [1..|V|]$.



■ **Figure 2** Cartesian trees of $T = 625178265$, $P = 7347$, $T[3..6] = 5178$, and $T[4..7] = 1782$. Here, $P \approx T[3..6]$ and $P \not\approx T[4..7]$.

Then $\text{insert}_V(i, c)$ inserts the symbol c at position i of V , $\text{delete}_V(k)$ deletes the k -th entry of V , $\text{rank}_c(V, j)$ returns the number of occurrences of c in $V[..j]$, $\text{rnkcnt}_V(i, j, c, d)$ returns $|\{x \in [i..j] \mid c \leq V[x] \leq d\}|$, $\text{select}_c(V, i)$ returns the index of the i -th occurrence of c in V if $i \leq \text{rank}_c(V, |V|)$, $\text{RNV}_V(i, j, c)$ returns the smallest value in $V[i..j]$ larger than c if it exists, and $\text{MI}_V(j, c)$ returns the maximal interval $[\ell..r]$ such that $0 \leq \ell \leq j \leq r \leq |V|$ and $V[x] \geq c$ for every $x \in [\ell + 1..r]$. See Figure 1 for examples. We represent strings by the following dynamic data structure, which supports the aforementioned operations and queries.

► **Lemma 2.1** ([17, Lemma 4]). *A dynamic string of length n over $[0..\sigma]$ with $\sigma \leq n^{O(1)}$ can be stored in a data structure occupying $O(n \lg \sigma)$ bits of space, supporting insertion, deletion and the queries access, rank, rnkcnt, select, RNV and MI in $O(\frac{\lg \sigma \lg n}{\lg \lg n})$ time.*

Alphabet. Throughout, we will work with the integer alphabet $\Sigma = [1..\sigma]$, where $\sigma \leq n^{O(1)}$, and a special symbol $\$ \notin \Sigma \cup \{0\}$ stipulated to be smaller than any symbol from Σ . The special symbol is motivated by the construction algorithm, and as a delimiter when a string should not be considered circular, as in the index of Kim and Cho [20]. Let $\Sigma_\$ = \Sigma \cup \{\$\}$.

Cartesian Tree Matching. The *Cartesian tree* $\text{ct}(V)$ of a string $V \in \Sigma_\* is a binary tree defined as follows. If $V = \varepsilon$, then $\text{ct}(V)$ is the empty tree. If $V \neq \varepsilon$, let i denote the position of the smallest symbol in V , where ties are broken with respect to text position. Then $\text{ct}(V)$ has i as its root, $\text{ct}(V[..i-1])$ as its left subtree and $\text{ct}(V[i+1..])$ as its right subtree. We say that two strings $U, V \in \Sigma_\* *Cartesian tree match* (*ct-match*) if and only if $\text{ct}(U) = \text{ct}(V)$, and write $U \approx V$. For instance, in Figure 2 the substring $T[3..6] = 5178$ of $T = 625178265$ ct-matches $P = 7347$ while $T[4..7] = 1782$ does not.

Parent Distance Encoding. Park et al. [32] use an encoding scheme for representing Cartesian trees that reduces the computation of a ct-match to checking whether the encoded strings exactly match. We here give a variant thereof which takes the special symbol $\$$ into account. Let $\infty \notin \Sigma_\$$ denote a symbol larger than any integer. The *parent distance encoding* $\langle V \rangle$ of $V \in \Sigma_\$$ is a string of length $|V|$ over $\Sigma_\$ \cup \{\infty\}$ such that

$$\langle V \rangle[i] = \begin{cases} i - \max\{j \in [..i-1] \mid V[j] \leq V[i]\} & \text{if } V[i] \neq \$ \text{ and such } j \text{ exists,} \\ V[i] & \text{if } V[i] = \$, \\ \infty & \text{otherwise,} \end{cases}$$

for each $i \in [1..|V|]$. We have $\langle V \rangle[..i] = \langle V[..i] \rangle$ for each $V \in \Sigma_{\S}^+$ and $i \in [1..|V|]$. For example, $\langle 41327\$3 \rangle = \text{xx}121\1 .²

► **Lemma 2.2** ([32, Section 2.4]). *Let $U, V \in \Sigma_{\S}^*$. The following statements are equivalent.*

- (i) $U \approx V$.
- (ii) $\langle U \rangle = \langle V \rangle$.
- (iii) $|U| = |V|$ and $U[i..j] \approx V[i..j]$ for each $1 \leq i \leq j \leq |U|$.
- (iv) $|U| = |V|$ and $\langle U \rangle[i..j] = \langle V \rangle[i..j]$ for each $1 \leq i \leq j \leq |U|$.

Problem Statement. We are interested in a solution to the following problem.

► **Problem (COUNT).** *Given $\emptyset \neq \mathcal{T} \subset \Sigma^+$ and $P \in \Sigma^*$, count each of the conjugates of the strings in $\mathcal{T}$ whose infinite iteration has a prefix ct-matching P .*

Throughout, let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma^+$. Our running example consists of the strings $T_1 = 512$, $T_2 = 5363$ and $T_3 = 4478$ over $\Sigma = [1..8]$. Given our running example, the solution to COUNT for $P_1 = 643$ and $P_2 = 5634$ is 0 and 2, respectively. Here, $P_2 \approx \text{Rot}(T_3, 2)^\omega[..4] \approx \text{Rot}(T_1, 2)^\omega[..4]$ since $\langle P_2 \rangle = \text{xx}1\text{xx}1 = \langle \text{Rot}(T_3, 2)^\omega[..4] \rangle = \langle \text{Rot}(T_1, 2)^\omega[..4] \rangle$.

3 $O(n \lg \sigma)$ -bit Index

Let $n = |T_1 \cdots T_d|$, $n_k = |T_k|$ for each $k \in [1..d]$, and $C_{\mathcal{T}}(i) = \text{Rot}(T_j, i - 1 - \sum_{k=1}^{j-1} n_k)$ for each $i \in [1..n]$, where $j = \min\{h \in [1..d] \mid \sum_{k=1}^h n_k \geq i\}$, i.e., we identify each conjugate of each text $T_1, \dots, T_d$ with its start position inside the concatenation $T_1 \cdots T_d$, such that we give them ranks from 1 to n . See Figure 3 for an example. In what follows, we put these ranks in a specific order by a permutation of $[1..n]$ such that the permuted ranks of all conjugates with prefixes of their infinite concatenation ct-matching a pattern form an interval $[\ell..r] \subseteq [1..n]$.

3.1 Conjugate Array

We express the permutation of the ranks of all conjugates by the so-called conjugate array, which we will subsequently define. To achieve this, we extend the ideas of Mantaci et al. [23] to Cartesian tree matching, and introduce a relation on $\Sigma_{\S}^+$. For notational convenience, we define the *rotational parent distance encoding* $\langle V \rangle_r$ of $V \in \Sigma_{\S}^+$ by $\langle V \rangle_r = \langle V^2 \rangle[|V| + 1..]$. For any $V, U \in \Sigma_{\S}^+$, let $V \preceq_{\omega} U$ if and only if there exists some natural number i such that $\langle V^\omega[..i] \rangle < \langle U^\omega[..i] \rangle$ or $\text{root}(\langle V \rangle_r) = \text{root}(\langle U \rangle_r)$ holds.

► **Lemma 3.1.** *The relation $\preceq_{\omega}$ defines a total preorder on $\Sigma_{\S}^+$, i.e., the relation $\preceq_{\omega}$ is binary, reflexive, transitive and connected.*

We call this preorder the ω -preorder. We write $V =_{\omega} U$ if and only if $V \preceq_{\omega} U \wedge U \preceq_{\omega} V$, and $V \prec_{\omega} U$ if and only if $V \preceq_{\omega} U \wedge V \neq_{\omega} U$. For instance, $T_3 = 4478 \prec_{\omega} 125 = \text{Rot}(T_1, 1)$ and $T_2 = 5363 =_{\omega} 6353 = \text{Rot}(T_2, 2)$. Note that the latter example violates antisymmetry, i.e., the relation $\preceq_{\omega}$ is not a total order.

Adapting Hon et al. [16, Lemma 5], we give a convenient but not necessarily optimal way to compute the ω -preorder of two given strings making use of the Weak Periodicity Lemma (cf. Crochemore and Rytter's book [4, Lemma 1.1]).

² As a side note, the parent distance encoding has been leveraged for devising compact $O(n)$ -bit data structures answering range minimum queries, such as the 2d-Min-Heap by Fischer [9] and the LRM-trees by Sadakane and Navarro [26], which semantically coincide.

► **Lemma 3.2** (Weak Periodicity Lemma). *Let p and q be two periods of a string X . If $p + q \leq |X|$, then $\gcd(p, q)$ is also a period of X .*

► **Lemma 3.3.** *Let $V, U \in \Sigma_{\mathbb{S}}^+$ and $z = \max\{|V|, |U|\}$. Then $V =_{\omega} U$ if and only if $\langle V^{\omega}[\dots 3z] \rangle = \langle U^{\omega}[\dots 3z] \rangle$.*

Proof. Without loss of generality, $|U| = z$. Let $|V| = i$, $|\text{root}(\langle V \rangle_r)| = j$ and $|\text{root}(\langle U \rangle_r)| = k$. ($\Leftarrow$) Assume $\langle V^{\omega}[\dots 3z] \rangle = \langle U^{\omega}[\dots 3z] \rangle$. Then, on the one hand,

$$\langle \text{Rot}(V, z) \rangle_r^{\omega}[\dots 2z] = \langle V^{\omega}[\dots 3z] \rangle[z + 1..] = \langle U^{\omega}[\dots 3z] \rangle[z + 1..] = \langle U \rangle_r \cdot \langle U \rangle_r$$

by Lemma 2.2. Since the rotational parent distance encoding is commutative with left rotations, j is a period of $\langle \text{Rot}(V, z) \rangle_r$. Consequently, both z and j are periods of $\langle U \rangle_r \cdot \langle U \rangle_r$. Since $j + z \leq 2z = |\langle U \rangle_r \cdot \langle U \rangle_r|$, we can apply Lemma 3.2 and find that $\gcd(j, z)$ is a period of $\langle U \rangle_r \cdot \langle U \rangle_r$. As $\gcd(j, z)$ divides $z = |\langle U \rangle_r|$, $\langle U \rangle_r$ can be formed by repeating $\langle U \rangle_r[\dots \gcd(j, z)]$ an integral number of times, which implies $\gcd(j, z) \geq k$, i.e., $i \geq j \geq k$. On the other hand,

$$\langle V \rangle_r \cdot \langle V \rangle_r = \langle V^{\omega}[\dots 3z] \rangle[i + 1..3i] = \langle U^{\omega}[\dots 3z] \rangle[i + 1..3i] = \langle \text{Rot}(U, i) \rangle_r^{\omega}[\dots 2i]$$

by Lemma 2.2. Since the rotational parent distance encoding is commutative with left rotations, k is a period of $\langle \text{Rot}(U, i) \rangle_r$. As $k \leq i$, k is a period of $\langle V \rangle_r \cdot \langle V \rangle_r$ in addition to i . Then $k + i \leq 2i = |\langle V \rangle_r \cdot \langle V \rangle_r|$ and Lemma 3.2 imply that $\gcd(k, i)$ is a period of $\langle V \rangle_r \cdot \langle V \rangle_r$. As $\gcd(k, i)$ divides $i = |\langle V \rangle_r|$, $\langle V \rangle_r[\dots \gcd(k, i)]$ can be repeated an integral number of times to form $\langle V \rangle_r$, which implies $\gcd(k, i) \geq j$. Consequently, $k \geq j$, and therefore $|\text{root}(\langle V \rangle_r)| = j = k = |\text{root}(\langle U \rangle_r)|$. In particular, $\text{root}(\langle V \rangle_r) = \langle V^{\omega}[\dots 3z] \rangle[3z - j + 1..] = \langle U^{\omega}[\dots 3z] \rangle[3z - k + 1..] = \text{root}(\langle U \rangle_r)$, i.e., $V =_{\omega} U$.

($\Rightarrow$) Let $V =_{\omega} U$. Assume $\langle V^{\omega}[\dots 3z] \rangle \neq \langle U^{\omega}[\dots 3z] \rangle$ with x minimal such that $\langle V^{\omega}[\dots 3z] \rangle[x] \neq \langle U^{\omega}[\dots 3z] \rangle[x]$. If $\max\{\langle V^{\omega}[\dots 3z] \rangle[x], \langle U^{\omega}[\dots 3z] \rangle[x]\} < \infty$, then $\text{Rot}(\text{root}(\langle V \rangle_r), x - 1)[1] = \langle V^{\omega}[\dots 3z] \rangle[x] \neq \langle U^{\omega}[\dots 3z] \rangle[x] = \text{Rot}(\text{root}(\langle U \rangle_r), x - 1)[1]$, which contradicts $V =_{\omega} U$. Hence, and without loss of generality, assume $\langle V^{\omega}[\dots 3z] \rangle[x] = \infty$. Then $\langle U^{\omega}[\dots 3z] \rangle[x] < \infty$, $x \leq |\text{root}(\langle V \rangle_r)|$, and consequently $\text{root}(\langle U \rangle_r)[x] = \langle U^{\omega}[\dots 3z] \rangle[x] < x \leq \text{root}(\langle V \rangle_r)[x]$, a contradiction to $V =_{\omega} U$. Thus, $\langle V^{\omega}[\dots 3z] \rangle = \langle U^{\omega}[\dots 3z] \rangle$. ◀

► **Remark 3.4.** As an alternative, the left direction of Lemma 3.3 can be proven by leveraging the Fine and Wilf theorem for exact string matching [8] as follows. We extend the definitions of the parent distance encoding and roots of a string to infinite strings over $\Sigma_{\mathbb{S}}$ and show that $U =_{\omega} V$ if and only if $\text{root}(\langle U^{\omega} \rangle[z + 1..]) = \text{root}(\langle V^{\omega} \rangle[z + 1..])$ for each $U, V \in \Sigma_{\mathbb{S}}^+$ with $z = \max\{|U|, |V|\}$. For $U, V \in \Sigma_{\mathbb{S}}^+$ and $z = \max\{|U|, |V|\}$, we then apply the Fine and Wilf theorem to the common prefix $\langle U^{\omega} \rangle[z + 1..3z] = \langle U^{\omega} \rangle[z + 1..3z] = \langle V^{\omega} \rangle[z + 1..3z] = \langle V^{\omega} \rangle[z + 1..3z]$ of $\langle U^{\omega} \rangle[z + 1..]$ and $\langle V^{\omega} \rangle[z + 1..]$ to obtain the desired result.

► **Corollary 3.5.** *Let $V, U \in \Sigma_{\mathbb{S}}^+$ and $z = \max\{|V|, |U|\}$. Then $V \prec_{\omega} U$ if and only if $\langle V^{\omega}[\dots 3z] \rangle < \langle U^{\omega}[\dots 3z] \rangle$.*

Similarly to Boucher et al. [2], we define the *conjugate array* $\text{CA}_{\mathcal{T}}$ of $\mathcal{T}$ as the string of length n over $[1..n]$ such that $\text{CA}_{\mathcal{T}}[i] = j$ if and only if

$$i - 1 = |\{k \in [1..n] \mid C_{\mathcal{T}}(k) \prec_{\omega} C_{\mathcal{T}}(j) \text{ or } C_{\mathcal{T}}(k) =_{\omega} C_{\mathcal{T}}(j) \wedge k < j\}|,$$

i.e., $i - 1$ is the number of all conjugates smaller than $C_{\mathcal{T}}(j)$ according to ω -preorder, where we break ties first with respect to text index, and then with respect to text position. By

i	$C_{\mathcal{T}}(i)$	$\langle C_{\mathcal{T}}(i)^{\omega}[\dots 12] \rangle$	$CA_{\mathcal{T}}^{-1}[i]$	$CA_{\mathcal{T}}[i]$	$\langle C_{\mathcal{T}}(CA_{\mathcal{T}}[i])^{\omega}[\dots 12] \rangle$	$C_{\mathcal{T}}(CA_{\mathcal{T}}[i])$
1	512	$\infty\infty 1131131131$	9	8	$\infty 11131113111$	4478
2	125	$\infty 11311311311$	3	9	$\infty 11311131113$	4784
3	251	$\infty 1\infty 113113113$	7	2	$\infty 11311311311$	125
4	5363	$\infty\infty 1212121212$	10	5	$\infty 12121212121$	3635
5	3635	$\infty 12121212121$	4	7	$\infty 12121212121$	3536
6	6353	$\infty\infty 1212121212$	11	10	$\infty 1\infty 111311131$	7844
7	3536	$\infty 12121212121$	5	3	$\infty 1\infty 113113113$	251
8	4478	$\infty 11131113111$	1	11	$\infty\infty 1113111311$	8447
9	4784	$\infty 11311131113$	2	1	$\infty\infty 1131131131$	512
10	7844	$\infty 1\infty 111311131$	6	4	$\infty\infty 1212121212$	5363
11	8447	$\infty\infty 1113111311$	8	6	$\infty\infty 1212121212$	6353

■ **Figure 3** The conjugate array $CA_{\mathcal{T}}$ of our running example $\mathcal{T} = \{512, 5363, 4478\}$.

resolving all ties this way, we ensure that $CA_{\mathcal{T}}$ is well-defined. Since $CA_{\mathcal{T}}$ is a permutation, its inverse, which we denote by $CA_{\mathcal{T}}^{-1}$, is also well-defined. See Figure 3 for our running example's conjugate array. We define the *conjugate range* $CR_{\mathcal{T}}(P)$ of a pattern $P \in \Sigma^*$ of length m in $\mathcal{T}$ as a maximal interval $[\ell..r] \subseteq [1..n]$ such that $P \approx C_{\mathcal{T}}(CA_{\mathcal{T}}[i])^{\omega}[\dots m]$ for every $i \in [\ell..r]$. Leveraging Lemma 2.2, we find that the conjugate range is well-defined.

► **Corollary 3.6.** *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma_s^+$, $n = |T_1 \dots T_d|$, $P \in \Sigma^*$, and $m = |P|$. Then $P \approx C_{\mathcal{T}}(CA_{\mathcal{T}}[i])^{\omega}[\dots m]$ if and only if $i \in CR_{\mathcal{T}}(P)$.*

We have $CR_{\mathcal{T}}(\varepsilon) = [1..n]$ because the empty string is a prefix of every conjugate. The computation of $CR_{\mathcal{T}}(P)$ for some pattern $P \in \Sigma^*$ on indexes related to the FM-index usually performs an algorithmic technique called the *backward search*. The backward search for P in $\mathcal{T}$ processes P backwards in an online manner and refines $CR_{\mathcal{T}}(P[i+1..])$ to $CR_{\mathcal{T}}(P[i..])$ at the i -th step on reading $P[i]$ with $P[|P|+1..] = \varepsilon$. Finally, the length of $CR_{\mathcal{T}}(P)$ is the solution to COUNT by Corollary 3.6. For our running example, $CR_{\mathcal{T}}(643) = \emptyset$ and $CR_{\mathcal{T}}(5634) = [6..7]$. Below we define the necessary tools to allow for an efficient backward search.

3.2 LF-mapping

We want to define a map that allows us to cycle backwards through each of the texts to be indexed by mapping to the position of a conjugate in the conjugate array from the position of its left rotation. However, if we want to represent this mapping space-efficiently, we have to be careful since we used tie-breaks within texts when we sorted the conjugates by a preorder. Our idea is to relax the requirements and define a map that allows us to cycle backwards through a text that is ω -equal to the original to dodge any issues arising from our tie-breaks. For that, we want to cycle backwards in text order inside the roots of each $\langle \dots \rangle_r$ -encoded text. Whenever we want to move backwards at the starting position of a root, we jump to its end position. We express this backwards movement with the following permutation. For every $i \in [1..n]$ with $j = \min\{h \in [1..d] \mid \sum_{k=1}^h n_k \geq i\}$, let

$$\text{prev}_{\mathcal{T}}(i) = \begin{cases} i - 1 + |\text{root}(\langle T_j \rangle_r)| & \text{if } C_{\mathcal{T}}(i) =_{\omega} T_j, \\ i - 1 & \text{otherwise.} \end{cases}$$

For our running example, $\text{prev}_{\mathcal{T}} = (1\ 3\ 2)(4\ 5)(6\ 7)(8\ 11\ 10\ 9)$. Here, we observe that we have two cycles $(4\ 5)$ and $(6\ 7)$ corresponding to $T_2[1..2] = 53$ and $T_2[3..4] = 63$, respectively, which

i	$T_3[i]$	$\text{Rot}(T_3, i)$	$\langle \text{Rot}(T_3, i) \rangle$	$\langle T_3[i] \cdot \text{Rot}(T_3, i) \rangle$	$\llbracket T_3 \rrbracket[i]$
1	4	4784	$\infty 113$	$\infty 1113$	1
2	4	7844	$\infty 1 \infty 1$	$\infty 1131$	2
3	7	8447	$\infty \infty 11$	$\infty 1 \infty 11$	1
4	8	4478	$\infty 111$	$\infty \infty 111$	0

■ **Figure 4** Rotational Cartesian tree signature encoding $\llbracket T_3 \rrbracket$ of $T_3 = 4478$.

are ω -equal to the original text $T_2 = 5363$. Now we are ready to express the LF-mapping in terms of the function $\text{prev}_{\mathcal{T}}$. The *LF-mapping* $\text{LF}_{\mathcal{T}}$ of $\mathcal{T}$ is a string of length n over $[1..n]$ such that $\text{LF}_{\mathcal{T}}[i] = \text{CA}_{\mathcal{T}}^{-1}[\text{prev}_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i])]$ for each $i \in [1..n]$. Since the LF-mapping is a permutation, it has an inverse $\text{LF}_{\mathcal{T}}^{-1}$, which we call the *FL-mapping* of $\mathcal{T}$ and denote by $\text{FL}_{\mathcal{T}}$. See Figure 5 for an example. The LF-mapping is at the core of the backward search. However, storing LF-mapping and FL-mapping in their plain form creates the need for two integer arrays of length n and entries of $\lg n$ bits, which motivates the following encoding scheme. Let the *rotational Cartesian tree signature encoding* $\llbracket V \rrbracket$ of $V \in \Sigma_{\S}^+$ denote a string of length $|V|$ over $\Sigma_{\S} \cup \{0\}$ such that

$$\llbracket V \rrbracket[i] = \begin{cases} V[i] & \text{if } V[i] = \$, \\ \text{rank}_{\infty}(\langle \text{Rot}(V, i) \rangle, |V|) - \text{rank}_{\infty}(\langle V[i] \cdot \text{Rot}(V, i) \rangle[2.., |V|]) & \text{otherwise,} \end{cases}$$

for each $i \in [1..|V|]$, i.e., if $V[i] \neq \$$, then $\llbracket V \rrbracket[i]$ reports the number of positions $j \in [1..|V|]$ such that $\text{Rot}(V, i)[j] \geq V[i]$ and $\langle \text{Rot}(V, i) \rangle[j] = \infty$. See Figure 4 for an example. Note that $\llbracket V \rrbracket[i.. \text{select}_{\S}(V, 1)] = \llbracket V[i.. \text{select}_{\S}(V, 1)] \rrbracket$ for each $V \in \Sigma_{\S}^+$ satisfying $\text{rank}_{\S}(V, |V|) \geq 1$ and $i \in [1.. \text{select}_{\S}(V, 1)]$.

► **Lemma 3.7.** *Let $V \in \Sigma_{\S}^+$. Then $\sum_{i=1}^j \llbracket V \rrbracket[i] \leq |V|$, where $j = \text{select}_{\S}(V, 1) - 1$ if $\text{rank}_{\S}(V, |V|) \geq 1$, and $j = |V|$ otherwise.*

Taking advantage of both encodings, we investigate how the ω -preorder of two strings changes if we rotate them. For convenience, we borrowed the following notation from literature [17, 14]. Let $\pi(V) = \llbracket V \rrbracket[1]$ for every $V \in \Sigma_{\S}^+$, and $\text{lcp}^{\infty}(U, W) = \text{rank}_{\infty}(\langle U \rangle, \text{lcp}(\langle U \rangle, \langle W \rangle))$ for each $U, W \in \Sigma_{\S}^*$. For example, $\pi(4478) = 1$ and $\text{lcp}^{\infty}(4478, 7844) = 1$.

► **Lemma 3.8** ([20, Lemma 3]). *Let $V, U \in \Sigma_{\S}^+$ such that $\text{Rot}(V, 1) \prec_{\omega} \text{Rot}(U, 1)$. Then $V \prec_{\omega} U$ if and only if (a) $\pi(V) = \$$ or (b) $\pi(V) \geq \min\{e, \pi(U)\}$ and $\pi(U) \neq \$$, where $e = \text{lcp}^{\infty}(\text{Rot}(V, 1), \text{Rot}(U, 1))$.*

Proof. We show the statement for $\min\{\pi(V), \pi(U)\} \in \Sigma$ since it is non-trivial. Let $z = \max\{|V|, |U|\}$ and $\lambda = \text{lcp}(\langle \text{Rot}(V, 1) \rangle^{\omega}[..3z], \langle \text{Rot}(U, 1) \rangle^{\omega}[..3z])$. Then $\langle \text{Rot}(V, 1) \rangle^{\omega}[..3z] < \langle \text{Rot}(U, 1) \rangle^{\omega}[..3z]$ by Corollary 3.5, $\lambda < 3z$, and $\text{select}_{\infty}(\langle \text{Rot}(V, 1) \rangle, e) \leq \min\{|V|, |U|\}$.

($\Rightarrow$) Assume $\pi(V) < \min\{e, \pi(U)\}$. Let $i = \text{select}_{\infty}(\langle V \rangle, 2)$. Then $\langle V \rangle[..i-1] = \langle U \rangle[..i-1]$ and $\langle V \rangle[i] = \infty \neq i-1 = \langle U \rangle[i]$, i.e. $\langle U \rangle < \langle V \rangle$. Consequently, $\langle U^{\omega}[..3z] \rangle < \langle V^{\omega}[..3z] \rangle$, which implies $U \prec_{\omega} V$ by Corollary 3.5. The statement follows by contraposition.

($\Leftarrow$) We exhaust all possible cases for $\pi(V) \geq \min\{e, \pi(U)\}$.

Case 1. Assume $\min\{e, \pi(V)\} > \pi(U)$. Let $i = \text{select}_{\infty}(\langle U \rangle, 2)$. Then $\langle V \rangle[..i-1] = \langle U \rangle[..i-1]$ and $\langle V \rangle[i] = i-1 \neq \infty = \langle U \rangle[i]$, i.e. $\langle V \rangle < \langle U \rangle$. Consequently, $\langle V^{\omega}[..3z] \rangle < \langle U^{\omega}[..3z] \rangle$, which implies $V \prec_{\omega} U$ by Corollary 3.5.

Case 2. Assume $\min\{\pi(V), \pi(U)\} \geq e$. Then we have $\langle V^{\omega}[..3z] \rangle[..\lambda+1] = \langle U^{\omega}[..3z] \rangle[..\lambda+1]$. Since $\text{Rot}(V, 1) \prec_{\omega} \text{Rot}(U, 1)$, $\lambda+2 \leq 3z$ by Lemma 3.3. If $\langle \text{Rot}(U, 1) \rangle^{\omega}[..3z][\lambda+1] = \infty$,

$\langle V^\omega[..\mathbf{3}z] \rangle[\lambda + 2] \leq \lambda < \lambda + 1 = \langle U^\omega[..\mathbf{3}z] \rangle[\lambda + 2]$. If $\langle \text{Rot}(U, 1)^\omega[..\mathbf{3}z] \rangle[\lambda] \neq \infty$, then $\langle V^\omega[..\mathbf{3}z] \rangle[\lambda + 2] = \langle \text{Rot}(V, 1)^\omega[..\mathbf{3}z] \rangle[\lambda + 1] < \langle \text{Rot}(U, 1)^\omega[..\mathbf{3}z] \rangle[\lambda + 1] = \langle U^\omega[..\mathbf{3}z] \rangle[\lambda + 2]$. Thus, $V \prec_\omega U$ by Corollary 3.5.

Case 3. Assume $e > \pi(V) = \pi(U)$. Then $\langle V^\omega[..\mathbf{3}z] \rangle[..\lambda + 1] = \langle U^\omega[..\mathbf{3}z] \rangle[..\lambda + 1]$ and $\lambda \leq z$. Moreover, $\langle V^\omega[..\mathbf{3}z] \rangle[\lambda + 2] = \langle \text{Rot}(V, 1)^\omega[..\mathbf{3}z] \rangle[\lambda + 1] < \langle \text{Rot}(U, 1)^\omega[..\mathbf{3}z] \rangle[\lambda + 1] = \langle U^\omega[..\mathbf{3}z] \rangle[\lambda + 2]$. Hence, $V \prec_\omega U$ by Corollary 3.5. $\blacktriangleleft$

Our representation of the LF- and FL-mapping consists of two strings $L_\mathcal{T}$ and $F_\mathcal{T}$, which are defined as follows. First, $L_\mathcal{T}$ is the string of length n over $\Sigma_\$ \cup \{0\}$ such that $L_\mathcal{T}[i] = \pi(C_\mathcal{T}(\text{CA}_\mathcal{T}[\text{LF}_\mathcal{T}[i]])) = \llbracket C_\mathcal{T}(\text{CA}_\mathcal{T}[i]) \rrbracket$ for each $i \in [1..n]$.

► **Corollary 3.9.** *Let $\emptyset \neq \mathcal{T} \subset \Sigma_\$^+$, n the accumulated length of all texts in $\mathcal{T}$, $i, j \in [1..n]$, and $i < j$. If $L_\mathcal{T}[i] = L_\mathcal{T}[j]$, then $\text{LF}_\mathcal{T}[i] < \text{LF}_\mathcal{T}[j]$.*

Second, $F_\mathcal{T}$ is the string of length n over $\Sigma_\$ \cup \{0\}$ such that $F_\mathcal{T}[\text{LF}_\mathcal{T}[i]] = L_\mathcal{T}[i]$ for each $i \in [1..n]$. By what follows, $L_\mathcal{T}$ and $F_\mathcal{T}$ suffice to compute both LF- and FL-mapping of $\mathcal{T}$.

► **Corollary 3.10.** *Let $\emptyset \neq \mathcal{T} \subset \Sigma_\$^+$, n the accumulated length of all texts in $\mathcal{T}$, and $i \in [1..n]$. Then $\text{LF}_\mathcal{T}[i] = \text{select}_{L_\mathcal{T}[i]}(F_\mathcal{T}, \text{rank}_{L_\mathcal{T}[i]}(L_\mathcal{T}, i))$ and $\text{FL}_\mathcal{T}[i] = \text{select}_{F_\mathcal{T}[i]}(L_\mathcal{T}, \text{rank}_{F_\mathcal{T}[i]}(F_\mathcal{T}, i))$.*

In Figure 5 we present $F_\mathcal{T}$ and $L_\mathcal{T}$ of our running example.

► **Remark 3.11.** Let $d = 1$, $\text{rank}_\$(T_1, n_1) = 1$ and $T_1[n_1] = \$$. If we substitute the occurrence of $\$$ in $F_\mathcal{T}$ and $L_\mathcal{T}$ for -1 , then the modified strings constitute the integer-based representation of Kim and Cho's index [20, Section 4].

3.3 Backward Search

The LF-mapping can be leveraged for the backward search by the following result, which is due to Lemma 2.2 and Corollary 3.6.

► **Lemma 3.12** ([20, Lemma 6]). *Let $\emptyset \neq \mathcal{T} \subset \Sigma_\$^+$, $P \in \Sigma^+$, $|P| = m$, $i \in [1..m]$, $h = \pi(P[i..] \cdot \$)$ and $e = \text{rank}_x(\langle P[i..] \rangle, m - i + 1)$. For $j \in \text{CR}_\mathcal{T}(P[i + 1..])$, $\text{LF}_\mathcal{T}[j] \in \text{CR}_\mathcal{T}(P[i..])$ if and only if (a) $e > 1$ and $L_\mathcal{T}[j] = h$ or (b) $e = 1$ and $L_\mathcal{T}[j] \geq h$.*

At this point, we can apply the techniques developed by Kim and Cho [20, Sections 5 and 6] to define an index of $\mathcal{T} \subset \Sigma^+$ that occupies $3n + o(n)$ bits of space and that solves COUNT in $O(m)$ time, where m is the pattern length. However, we defer the compact representation to Section 5. That is because we here focus on a space-efficient construction of our proposed index and its extension, for which we will represent $F_\mathcal{T}$ and $L_\mathcal{T}$ by the dynamic data structure of Lemma 2.1, and introduce an auxiliary string for the backward search. Let $\text{LCP}_\mathcal{T}^\infty$ denote a string of length n over Σ such that $\text{LCP}_\mathcal{T}^\infty[1] = 0$ and $\text{LCP}_\mathcal{T}^\infty[i] = \text{lcp}^\infty(C_\mathcal{T}(\text{CA}_\mathcal{T}[i]), C_\mathcal{T}(\text{CA}_\mathcal{T}[i - 1]))$ for each $i \in [2..n]$.

► **Lemma 3.13.** *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma_\$^+$, $n = |T_1 \cdots T_d|$, $i, j \in [1..n]$, and $i < j$. Then $\text{lcp}^\infty(C_\mathcal{T}(\text{CA}_\mathcal{T}[i]), C_\mathcal{T}(\text{CA}_\mathcal{T}[j])) = \min\{\text{LCP}_\mathcal{T}^\infty[k] \mid k \in [i + 1..j]\} = \text{RNV}_{\text{LCP}_\mathcal{T}^\infty}(i + 1, j, -1)$.*

Proof. Let $U, V, W \in \Sigma_\$^+$ and $z = \max\{|U|, |V|, |W|\}$. Then $\text{lcp}(\langle U^\omega[..\mathbf{3}z] \rangle, \langle W^\omega[..\mathbf{3}z] \rangle) = \min\{\text{lcp}(\langle U^\omega[..\mathbf{3}z] \rangle, \langle V^\omega[..\mathbf{3}z] \rangle), \text{lcp}(\langle V^\omega[..\mathbf{3}z] \rangle, \langle W^\omega[..\mathbf{3}z] \rangle)\}$ if we have $\langle U^\omega[..\mathbf{3}z] \rangle \leq \langle V^\omega[..\mathbf{3}z] \rangle$ and $\langle V^\omega[..\mathbf{3}z] \rangle \leq \langle W^\omega[..\mathbf{3}z] \rangle$. Then the statement follows by Lemma 3.3 and the definition of $\text{LCP}_\mathcal{T}^\infty$. $\blacktriangleleft$

► **Lemma 3.14.** *Let $\emptyset \neq \mathcal{T} \subset \Sigma_\$^+$, $P \in \Sigma^+$, $m = |P|$, $i \in [1..m]$, $[\ell..r] = \text{CR}_\mathcal{T}(P[i + 1..])$, $h = \pi(P[i..] \cdot \$)$, and $e = \text{rank}_x(\langle P[i..] \rangle, m - i + 1)$. Then Algorithm 1 correctly computes $[\ell'..r'] = \text{CR}_\mathcal{T}(P[i..])$.*

■ **Algorithm 1** Computing the conjugate range $\text{CR}_{\mathcal{T}}(P[i..])$. Here, $\emptyset \neq \mathcal{T} \subset \Sigma_{\mathcal{S}}^+$, $P \in \Sigma^+$, $m = |P|$, $i \in [1..m]$, $[\ell..r] = \text{CR}_{\mathcal{T}}(P[i+1..])$, $h = \pi(P[i..]\$)$, and $e = \text{rank}_{\infty}(\langle P[i..] \rangle, m-i+1)$.

```

1 Function crangeupd( $e, h, [\ell..r], \text{L}_{\mathcal{T}}, \text{F}_{\mathcal{T}}, \text{LCP}_{\mathcal{T}}^{\infty}$ ):
2    $r' \leftarrow r$ ;
3   if  $e > 1$  then                                     // Case 1 in Lemma 3.14
4     if  $c \leftarrow \text{rkcnt}_{\text{L}_{\mathcal{T}}}(\ell, r, h, h) \geq 1$  then
5        $r' \leftarrow \text{LF}_{\mathcal{T}}[\text{select}_h(\text{L}_{\mathcal{T}}, \text{rank}_h(\text{L}_{\mathcal{T}}, r))]$ ;
6   else                                                 // Case 2 in Lemma 3.14
7     if  $c \leftarrow \text{rkcnt}_{\text{L}_{\mathcal{T}}}(\ell, r, h, \sigma) \geq 1$  then
8        $v \leftarrow \text{RNV}_{\text{L}_{\mathcal{T}}}(\ell, r, h-1)$ ;
9        $x \leftarrow \text{select}_v(\text{L}_{\mathcal{T}}, \text{rank}_v(\text{L}_{\mathcal{T}}, r))$ ;
10       $[\ell''..r''] \leftarrow \text{MI}_{\text{LCP}_{\mathcal{T}}^{\infty}}(x, v+1)$ ;
11       $y \leftarrow \text{rkcnt}_{\text{L}_{\mathcal{T}}}(\ell, x-1, h, \sigma) + \text{rkcnt}_{\text{L}_{\mathcal{T}}}(x+1, r'', h, \sigma)$ ;
12       $r' \leftarrow \text{LF}_{\mathcal{T}}[x] + c - (y+1)$ ;
13  return  $[r' - c + 1..r']$ ;                             // returns  $[r+1..r] = \emptyset$  if  $c = 0$ 

```

Proof. Let $c = |\text{CR}_{\mathcal{T}}(P[i..])|$. We show how Algorithm 1 computes c and r' to obtain $[\ell'..r']$.

Case 1. Assume $e > 1$. This case is handled from Line 3 through Line 5. By Lemma 3.12, $\text{LF}_{\mathcal{T}}[j] \in \text{CR}_{\mathcal{T}}(P[i..])$ if and only if $\text{L}_{\mathcal{T}}[j] = h$ for each $j \in [\ell..r]$. We compute c in Line 4 and return an empty interval in Line 13 due to Line 2 if $c \leq 0$. Assume $c \geq 1$. We compute the largest $x \in [\ell..r]$ such that $\text{L}_{\mathcal{T}}[x] = h$, and then $r' = \text{LF}_{\mathcal{T}}[x]$ in Line 5, with correctness following by Corollary 3.9.

Case 2. Assume $e = 1$. This case is handled from Line 6 through Line 12. By Lemma 3.12, $\text{LF}_{\mathcal{T}}[j] \in \text{CR}_{\mathcal{T}}(P[i..])$ if and only if $\text{L}_{\mathcal{T}}[j] \geq h$ for each $j \in [\ell..r]$. Thus, we correctly compute c in Line 7, and return an empty interval in Line 13 due to Line 2 if $c \leq 0$. Assume $c \geq 1$. We compute the lowest value v in $\text{L}_{\mathcal{T}}[\ell..r]$ greater than $h-1$ in Line 8 and determine the largest $x \in [\ell..r]$ such that $\text{L}_{\mathcal{T}}[x] = v$ in Line 9. Let $[\ell''..r'']$ denote the interval computed in Line 10. Then $\text{lcp}^{\infty}(C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[j]), C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[k])) \geq v+1$ for each $j, k \in [\ell''..r'']$ by Lemma 3.13. The following statements hold due to Lemma 3.8.

- If $j \in [\ell..x-1]$ satisfies $\text{L}_{\mathcal{T}}[j] \geq h$, then $\text{LF}_{\mathcal{T}}[j] < \text{LF}_{\mathcal{T}}[x]$ since $v \leq \text{L}_{\mathcal{T}}[j]$.
- If $j \in [x+1..r'']$ satisfies $\text{L}_{\mathcal{T}}[j] \geq h$, then $\text{LF}_{\mathcal{T}}[j] < \text{LF}_{\mathcal{T}}[x]$ since $v < \text{L}_{\mathcal{T}}[j]$ and $v < v+1 \leq \text{lcp}^{\infty}(C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[j]), C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[x]))$.
- If $j \in [r''+1..r]$ satisfies $\text{L}_{\mathcal{T}}[j] \geq h$, then $\text{LF}_{\mathcal{T}}[j] > \text{LF}_{\mathcal{T}}[x]$ since we have $v \geq \text{lcp}^{\infty}(C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[j]), C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[x]))$.

We apply these statements to compute $y = \text{LF}_{\mathcal{T}}[x] - \ell'$ in Line 11, and then infer $r' = \text{LF}_{\mathcal{T}}[x] + c - (y+1)$ in Line 12. ◀

The next result is obtained from the computation of Demaine and colleagues' [5] Cartesian tree signature encoding of the given string.

► **Lemma 3.15.** *Given $P \in \Sigma_{\mathcal{S}}^*$ of length m , we can process P in $O(m)$ time such that we can subsequently compute $\pi(P[i..]\$)$ and $\text{rank}_{\infty}(P[i..], m-i+1)$ in $O(1)$ time, for every $i \in [1..m]$.*

► **Theorem 3.16.** *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma_{\mathcal{S}}^+$ and $n = |T_1 \cdots T_d|$. There exists a data structure occupying $O(n \lg \sigma)$ bits of space that solves COUNT in $O(m \frac{\lg \sigma \lg n}{\lg \lg n})$ time, where m is the pattern length.*

i	$CA_{\mathcal{T}}[i]$	$C_{\mathcal{T}}(CA_{\mathcal{T}}[i])$	$LF_{\mathcal{T}}[i]$	$F_{\mathcal{T}}[i]$	$L_{\mathcal{T}}[i]$	$LCP_{\mathcal{T}}^{\infty}[i]$	$\langle C_{\mathcal{T}}(CA_{\mathcal{T}}[i]) \rangle$
1	8	4478	8	1	0	0	$\infty 111$
2	9	4784	1	2	1	1	$\infty 113$
3	2	125	9	2	0	1	$\infty 11$
4	5	3635	10	2	0	1	$\infty 121$
5	7	3536	11	2	0	1	$\infty 121$
6	10	7844	2	1	2	1	$\infty 1 \infty 1$
7	3	251	3	1	2	2	$\infty 1 \infty$
8	11	8447	6	0	1	1	$\infty \infty 11$
9	1	512	7	0	1	2	$\infty \infty 1$
10	4	5363	4	0	2	2	$\infty \infty 12$
11	6	6353	5	0	2	2	$\infty \infty 12$

■ **Figure 5** The ceBWT index of our running example $\mathcal{T} = \{512, 5363, 4478\}$.

Proof. We represent $F_{\mathcal{T}}$, $L_{\mathcal{T}}$ and $LCP_{\mathcal{T}}^{\infty}$ by the data structure of Lemma 2.1, which leads to the claimed space complexity. We preprocess a pattern $P \in \Sigma^+$ of length m with Lemma 3.15, and compute $CR_{\mathcal{T}}(P[i..])$ from $CR_{\mathcal{T}}(P[i+1..])$ for each $i \in [1..m]$ in descending order leveraging Lemma 3.14. Since each conjugate range update takes $O(1)$ queries, the claimed complexity for solving COUNT follows from Lemma 2.1. ◀

We call the data structure of Theorem 3.16 the *ceBWT index* of $\mathcal{T}$. The ceBWT index of the running example is presented in Figure 5.

► **Remark 3.17 (Locating Occurrences).** In case we want to report the locations of occurrences, we can construct $CA_{\mathcal{T}}$ alongside the ceBWT index and store it in $O(n + d \lg n)$ bits of space by sampling each $(\lg n)$ -th text position [7, Sect. 3.2], where the latter term is due to the d cycles of the LF-mapping corresponding to the $\langle \cdot \rangle_r$ -encoded roots of the input texts, which require at least a sample each. We maintain an auxiliary bit string indicating the sampled locations in lex-order, and represent the sampled conjugate array, its auxiliary bit string, $F_{\mathcal{T}}$ and a copy of $L_{\mathcal{T}}$ by the data structure presented in Lemma 6.5. Then accessing the sampled conjugate array, checking if a position is sampled, and computing the LF- and FL-mapping takes $O(\frac{\lg n}{\lg \lg n})$ time. Consequently, we can answer locate queries with the dynamic variant of the ceBWT index in $O(m \frac{\lg \sigma \lg n}{\lg \lg n} + occ \frac{\lg^2 n}{\lg \lg n})$ time, where m is the pattern length and occ the number of occurrences.

3.4 Showcase Backward Search

We give an example of the backward search for $P = 375 \in \Sigma^+$ in our running example $\mathcal{T} = \{512, 5363, 4478\}$ leveraging the ceBWT index of $\mathcal{T}$, which is presented in Figure 5 and Figure 6.

Since ε is a prefix of every conjugate considered in $\mathcal{T}$, initially $[\ell..r] = CR_{\mathcal{T}}(P[4..]) = CR_{\mathcal{T}}(\varepsilon) = [1..11]$. We observe that there exists no symbol smaller than or equal to $P[3] = 5$ in $P[4..] = \varepsilon$, i.e., $e = \text{rank}_x(\langle P[3..] \rangle, 3 - 3 + 1) = 1$. Consequently, we enter Line 6 of Algorithm 1 for the conjugate range update of $CR_{\mathcal{T}}(P[4..])$ to $CR_{\mathcal{T}}(P[3..])$. Then Lemma 3.12 tells us that the conjugates corresponding to entries $i \in [\ell..r] = [1..11]$ with $L_{\mathcal{T}}[i] \geq h = \pi(P[3..] \cdot \$) = 0$ are the left rotations of conjugates whose prefix of length 1 ct-matches $P[3..] = 5$. Thus, we compute $c = \text{rnkcnt}_{L_{\mathcal{T}}}(1, 11, 0, 8) = 11$ to find the length of $CR_{\mathcal{T}}(P[3..])$. Since the length is positive, it remains to compute either the left or the right boundary of the conjugate range and

infer the respective other from its length. To that end, we compute $v = \text{RNV}_{L_{\mathcal{T}}}(1, 11, -1) = 0$, $x = \text{select}_0(L_{\mathcal{T}}, \text{rank}_0(L_{\mathcal{T}}, 11)) = 5$ and $[\ell''..r''] = \text{MI}_{\text{LCP}_{\mathcal{T}}}^{\infty}(5, 0 + 1) = [1..11]$, which are the lowest value in $L_{\mathcal{T}}[1..11]$ not smaller than $h = 0$, the highest index in $[1..11]$ at which the value v occurs and the interval whose entries correspond to the parent distance encoded conjugates that have more than v occurrences of ∞ in the longest common prefix with the parent distance encoded conjugate corresponding to the index x , respectively. With v , x and $[\ell''..r'']$ we can leverage Lemma 3.8 as shown in the proof of Lemma 3.14 to compute the number y of entries $j \in [1..11] - \{11\}$ with $L_{\mathcal{T}}[j] \geq h = 0$ such that $C_{\mathcal{T}}(\text{LF}_{\mathcal{T}}[j]) \preceq_{\omega} C_{\mathcal{T}}(\text{LF}_{\mathcal{T}}[x])$, i.e., $\text{LF}_{\mathcal{T}}[x]$ is the $(y+1)$ -smallest element of $\text{CR}_{\mathcal{T}}(P[3..])$. We obtain $y = \text{rnkcnt}_{L_{\mathcal{T}}}(1, 5 - 1, 0, 8) + \text{rnkcnt}_{L_{\mathcal{T}}}(5 + 1, 11, 0, 8) = 4 + 6 = 10$ and subsequently infer $r' = \text{LF}_{\mathcal{T}}[5] + 11 - (10 + 1) = 11 + 11 - 11 = 11$ and $\ell' = 11 - 11 + 1 = 1$. Thus, $[\ell'..r'] = \text{CR}_{\mathcal{T}}(P[3..]) = [1..11]$, which one can confirm by, e.g., looking at the last column of Figure 5 or Figure 6.

Next, we set $[\ell..r] = [\ell'..r'] = [1..11]$ and compute $\text{CR}_{\mathcal{T}}(P[2..]) = \text{CR}_{\mathcal{T}}(75)$. We observe that there exist symbols smaller than $P[2] = 7$ in $P[3..] = 5$, i.e., $e = \text{rank}_x(\langle P[2..] \rangle, 3 - 2 + 1) = 2$. Consequently, we enter Line 3 of Algorithm 1 for the update of $\text{CR}_{\mathcal{T}}(P[3..])$ to $\text{CR}_{\mathcal{T}}(P[2..])$. Then Lemma 3.12 tells us that the conjugates corresponding to entries $i \in [\ell..r] = [1..11]$ with $L_{\mathcal{T}}[i] = h = \pi(P[2..] \cdot \$) = 0$ are the left rotations of conjugates whose prefix of length 2 ct-matches $P[2..] = 5$. We compute the length $c = \text{rnkcnt}_{L_{\mathcal{T}}}(1, 11, 0, 0) = 4$ of $\text{CR}_{\mathcal{T}}(P[2..])$ and find that it is positive. Then we leverage Corollary 3.9 to compute the right boundary $r' = \text{LF}_{\mathcal{T}}[\text{select}_0(L_{\mathcal{T}}, 11)] = \text{LF}_{\mathcal{T}}[5] = 11$ of $\text{CR}_{\mathcal{T}}(P[2..])$, and infer $\ell' = 11 - 4 + 1 = 8$ from its length c , i.e., $[\ell'..r'] = \text{CR}_{\mathcal{T}}(P[2..]) = [8..11]$.

Last, we set $[\ell..r] = [\ell'..r'] = [8..11]$ for the computation of $\text{CR}_{\mathcal{T}}(P[1..]) = \text{CR}_{\mathcal{T}}(375)$. We observe that there occurs no symbol smaller than or equal to $P[1] = 3$ in $P[2..] = 75$, i.e., $e = \text{rank}_x(\langle P[1..] \rangle, 3 - 3 + 1) = 1$. Thus, we enter Line 6 of Algorithm 1 and proceed similarly to the first conjugate range update. We compute $h = \pi(P[1..] \cdot \$) = 2$ and find that $|\text{CR}_{\mathcal{T}}(P)| = c = \text{rnkcnt}_{L_{\mathcal{T}}}(8, 11, 2, 8) = 2$ is positive, i.e., the conjugate range is not empty. Then $v = \text{RNV}_{L_{\mathcal{T}}}(8, 11, 1) = 2$, $x = \text{select}_2(L_{\mathcal{T}}, \text{rank}_2(L_{\mathcal{T}}, 11)) = 11$, $[\ell''..r''] = \text{MI}_{\text{LCP}_{\mathcal{T}}}^{\infty}(11, 3) = [11..11]$, $y = \text{rnkcnt}_{L_{\mathcal{T}}}(8, 10, 2, 8) + \text{rnkcnt}_{L_{\mathcal{T}}}(12, 11, 2, 8) = 1 + 0 = 1$, $r' = \text{LF}_{\mathcal{T}}[11] + 2 - (1 + 1) = 5$, and $\ell' = 5 - 2 + 1 = 4$, i.e., $\text{CR}_{\mathcal{T}}(P) = \text{CR}_{\mathcal{T}}(P[1..]) = [4..5]$.

i	$C_{\mathcal{T}}(i)$	$\langle C_{\mathcal{T}}(i)^{\omega}[\cdot, 12] \rangle$	$CA_{\mathcal{T}}^{-1}[i]$	$CA_{\mathcal{T}}[i]$	$C_{\mathcal{T}}(CA_{\mathcal{T}}[i])$	$FL_{\mathcal{T}}[i]$	$F_{\mathcal{T}}[i]$	$\llbracket C_{\mathcal{T}}(CA_{\mathcal{T}}[i]) \rrbracket$	$L_{\mathcal{T}}[i]$	$LF_{\mathcal{T}}[i]$	$LCP_{\mathcal{T}}^{\infty}[i]$	$\langle C_{\mathcal{T}}(CA_{\mathcal{T}}[i])^{\omega}[\cdot, 12] \rangle$
1	512	$\alpha\alpha\alpha 1131131131$	9	8	4478	2	1	1210	0	8	0	$\alpha 11131113111$
2	125	$\alpha 11311311311$	3	9	4784	6	2	2101	1	1	1	$\alpha 11311131113$
3	251	$\alpha 1\alpha 113113113$	7	2	125	7	2	210	0	9	1	$\alpha 11311311311$
4	5363	$\alpha\alpha\alpha 1212121212$	10	5	3635	10	2	2020	0	10	1	$\alpha 12121212121$
5	3635	$\alpha 12121212121$	4	7	3536	11	2	2020	0	11	1	$\alpha 12121212121$
6	6353	$\alpha\alpha\alpha 1212121212$	11	10	7844	8	1	1012	2	2	1	$\alpha 1\alpha 111311131$
7	3536	$\alpha 12121212121$	5	3	251	9	1	102	2	3	2	$\alpha 1\alpha 113113113$
8	4478	$\alpha 113113111$	1	11	8447	1	0	0121	1	6	1	$\alpha\alpha\alpha 1113111311$
9	4784	$\alpha 1131131113$	2	1	512	3	0	021	1	7	2	$\alpha\alpha\alpha 1131131131$
10	7844	$\alpha 1\alpha 111311131$	6	4	5363	4	0	0202	2	4	2	$\alpha\alpha\alpha 1212121212$
11	8447	$\alpha 1\alpha 111311131$	8	6	6353	5	0	0202	2	5	2	$\alpha\alpha\alpha 1212121212$

Figure 6 Data structures of our running example $\mathcal{T} = \{512, 5363, 4478\}$.

4 Construction in $O(n \lg \sigma)$ Bits of Space

We will show how to construct the ceBWT index of a single text, and how an existing ceBWT index can be extended to also index an additional text. We then leverage these two results to iteratively construct the ceBWT index of any set of texts, adding one new text per iteration step.

4.1 Single Text ceBWT Index

We need another technical result before we can tackle the construction of the ceBWT index of a single text.

► **Lemma 4.1.** *Let $V, U \in \Sigma_{\$}^+$, $\text{Rot}(V, 1) \prec_{\omega} \text{Rot}(U, 1)$, and $e = \text{lcp}^{\infty}(\text{Rot}(V, 1), \text{Rot}(U, 1))$. Then*

$$\text{lcp}^{\infty}(V, U) = \begin{cases} 0 & \text{if } \pi(U) = \$ \text{ or } \pi(V) = \$, \\ e - \pi(V) + 1 & \text{if } V \prec_{\omega} U \text{ and } \$ \neq \pi(V) = \pi(U) < e, \\ 1 & \text{otherwise.} \end{cases}$$

Proof. If $\pi(W) = \$$ for any $W \in \Sigma_{\$}^+$, then $\text{rank}_x(\langle W \rangle, |W|) = 0$. It remains to show the cases where both $\pi(V)$ and $\pi(U)$ do not evaluate to $\$$. Thus, assume $\$ < \min\{\pi(V), \pi(U)\}$, i.e., $\text{lcp}^{\infty}(U, V) \geq 1$. By assumption, $V \neq_{\omega} U$.

Case 1. Assume $U \prec_{\omega} V$. Then $\pi(V) < e$ and $\pi(V) < \pi(U)$ by Lemma 3.8. Consequently, $\langle V \rangle[\text{select}_x(\langle V \rangle, 2)] = \infty > \text{select}_x(\langle V \rangle, 2) - 1 = \langle U \rangle[\text{select}_x(\langle V \rangle, 2)]$, i.e., $\text{lcp}^{\infty}(V, U) = 1$.

Case 2 Assume $V \prec_{\omega} U$. Then $\pi(V) \geq e$ or $\pi(V) \geq \pi(U)$ by Lemma 3.8.

Case 2.1 Assume $e \geq \pi(U) = \pi(V)$. Then $\text{lcp}(\langle V \rangle, \langle U \rangle) = \text{lcp}(\langle \text{Rot}(V, 1) \rangle, \langle \text{Rot}(U, 1) \rangle) + 1$. Hence, $\text{lcp}^{\infty}(V, U) = e - \pi(V) + 1$, which evaluates to 1 if and only if $e = \pi(V) = \pi(U)$.

Case 2.2 Assume $\min\{e, \pi(V)\} > \pi(U)$. Then $\langle V \rangle[\text{select}_x(\langle U \rangle, 2)] = \text{select}_x(\langle U \rangle, 2) - 1 < \infty = \langle U \rangle[\text{select}_x(\langle U \rangle, 2)]$. Consequently, $\text{lcp}^{\infty}(V, U) = 1$.

Case 2.3 Assume $e \leq \min\{\pi(V), \pi(U)\}$. If $\text{lcp}(\langle \text{Rot}(V, 1) \rangle, \langle \text{Rot}(U, 1) \rangle) = \min\{|\langle U \rangle|, |\langle V \rangle|\}$, then $\text{lcp}^{\infty}(V, U) = \text{rank}_x(\langle V \rangle, \min\{|\langle V \rangle|, |\langle U \rangle|\}) = 1$. Thus, assume $\min\{|\langle U \rangle|, |\langle V \rangle|\} > \text{lcp}(\langle \text{Rot}(V, 1) \rangle, \langle \text{Rot}(U, 1) \rangle)$. Then there exists some $x \in [1.. \min\{|\langle U \rangle|, |\langle V \rangle|\} - 1]$ such that $\langle \text{Rot}(V, 1) \rangle[..x - 1] = \langle \text{Rot}(U, 1) \rangle[..x - 1]$ and $\text{Rot}(V, 1)[x] < \text{Rot}(U, 1)[x]$. If $\text{Rot}(U, 1)[x] = \infty$, then $\langle U \rangle[x + 1] \geq x > \langle V \rangle[x + 1]$. If $\text{Rot}(U, 1)[x] \neq \infty$, then $\langle U \rangle[x + 1] = \text{Rot}(U, 1)[x] > \text{Rot}(V, 1)[x] = \langle V \rangle[x + 1]$. Either way, $\text{lcp}^{\infty}(V, U) = \text{rank}_x(\langle V \rangle, x) = 1$. ◀

We will first show how to construct the ceBWT index of a text from $\Sigma_{\$}^+$ in which $\$$ occurs exactly once, and then show how to construct the ceBWT index of an arbitrary text from Σ^+ (without the requirement on $\$$).

► **Lemma 4.2.** *Let $R \in \Sigma_{\$}^+$, $\rho = |R| \geq 2$, $R[\rho] = \$$, and $\text{rank}_{\$}(R, \rho) = 1$. Algorithm 2 correctly computes $\text{CA}_{\{bR\}}^{-1}[1]$ and the ceBWT index of $\{bR\}$ for each $b \in \Sigma$.*

Proof. For each $i \in [0.. \pi(bR)]$, let $J_i = [\ell_i..r_i]$ maximal such that $r_i \leq \text{CA}_{\{R\}}^{-1}[1]$ and $\text{lcp}^{\infty}(R, C_{\{R\}}(\text{CA}_{\{R\}}[j])) = i$ for each $j \in J_i$, and $J_{\pi(bR)+1} = [\ell_{\pi(bR)+1}..r_{\pi(bR)+1}]$ maximal such that $\text{lcp}^{\infty}(R, C_{\{R\}}(\text{CA}_{\{R\}}[j])) \geq \pi(bR) + 1$ for each $j \in J_{\pi(bR)+1}$, i.e., the left and right boundary of the former are computed in Line 6 and Line 5, respectively, and the latter in Line 2. The following statements are due to the location of the single $\$$ in each conjugate of R and Lemma 3.8.

■ **Algorithm 2** Computing $c + 1 = \text{CA}_{\{bR\}}^{-1}[1]$ and updating the ceBWT of $\{R\}$ to that of $\{bR\}$ for $b \in \Sigma$. Here, $R \in \Sigma_{\$}^{+}$, $\rho = |R|$, $R[\rho] = \$$, $\text{rank}_{\$}(R, \rho) = 1$, and $y = \text{CA}_{\{R\}}^{-1}[1]$.

```

1 Function extend( $\pi(bR), y, F_{\{R\}}, L_{\{R\}}, \text{LCP}_{\{R\}}^{\infty}$ ):
2    $[\ell..r] \leftarrow \text{MI}_{\text{LCP}_{\{R\}}^{\infty}}(y, \pi(bR) + 1)$ ;
3    $c \leftarrow \text{rnkcnt}_{L_{\{R\}}}(\ell, y - 1, \pi(bR), \sigma) + 1 + \text{rnkcnt}_{L_{\{R\}}}(y + 1, r, \pi(bR) + 1, \sigma)$ ;
4   for  $i \leftarrow \pi(bR)$  to 0 do
5      $r' \leftarrow \ell - 1$ ;
6      $[\ell..r] \leftarrow \text{MI}_{\text{LCP}_{\{R\}}^{\infty}}(y, i)$ ;
7      $c \leftarrow c + \text{rnkcnt}_{L_{\{R\}}}(\ell, r', i, \sigma)$ ;
8    $p, s \leftarrow 1$ ;           //  $p$  stores  $\text{LCP}_{\{bR\}}^{\infty}[c + 1]$ , and, if  $c < \rho$ ,  $s$  stores  $\text{LCP}_{\{bR\}}^{\infty}[c + 2]$ 
9   if  $\text{FL}_{\{R\}}[c] < y$  and  $\pi(bR) = F_{\{R\}}[c]$  then
10     $p \leftarrow \max\{\text{RNV}_{\text{LCP}_{\{R\}}^{\infty}}(\text{FL}_{\{R\}}[c] + 1, y, -1), \pi(bR)\} - \pi(bR) + 1$ ;
11  if  $c < \rho$  and  $\text{FL}_{\{R\}}[c + 1] > y$  and  $\pi(bR) = F_{\{R\}}[c + 1]$  then
12     $s \leftarrow \max\{\text{RNV}_{\text{LCP}_{\{R\}}^{\infty}}(y + 1, \text{FL}_{\{R\}}[c + 1], -1), \pi(bR)\} - \pi(bR) + 1$ ;
13   $\text{insert}_{\text{LCP}_{\{R\}}^{\infty}}(c + 1, p)$ ;
14  if  $c < \rho$  then  $\text{LCP}_{\{R\}}^{\infty}[c + 2] \leftarrow s$ ;
15   $\text{insert}_{F_{\{R\}}}(c + 1, \pi(bR))$ ;
16   $L_{\{R\}}[y] \leftarrow \pi(bR)$ ;
17   $\text{insert}_{L_{\{R\}}}(c + 1, \$)$ ;
18  return  $c + 1, F_{\{R\}}, L_{\{R\}}, \text{LCP}_{\{R\}}^{\infty}$ ;           //  $\text{CA}_{\{bR\}}^{-1}[1]$  and the ceBWT index of  $\{bR\}$ 

```

- If $j \in [r_{\pi(bR)+1} + 1.. \rho]$, then $bR \prec_{\omega} C_{\{R\}}(\text{CA}_{\{R\}}[\text{LF}_{\{R\}}[j]])$.
- If $j \in [\text{CA}_{\{R\}}^{-1}[1] + 1.. r_{\pi(bR)+1}]$, then $C_{\{R\}}(\text{CA}_{\{R\}}[\text{LF}_{\{R\}}[j]]) \prec_{\omega} bR$ if and only if $L_{\{R\}}[j] \geq \pi(bR) + 1$.
- If $j = \text{CA}_{\{R\}}^{-1}[1]$, then $C_{\{R\}}(\text{CA}_{\{R\}}[\text{LF}_{\{R\}}[j]]) = (\$ \cdot R)[.. \rho] \prec_{\omega} bR$.
- If $j \in [\ell_{\pi(bR)+1}.. \text{CA}_{\{R\}}^{-1}[1] - 1]$, then $C_{\{R\}}(\text{CA}_{\{R\}}[\text{LF}_{\{R\}}[j]]) \prec_{\omega} bR$ if and only if $L_{\{R\}}[j] \geq \pi(bR)$.
- If $i \in [0.. \pi(bR)]$ and $j \in J_i$, then $C_{\{R\}}(\text{CA}_{\{R\}}[\text{LF}_{\{R\}}[j]]) \prec_{\omega} bR$ if and only if $L_{\{R\}}[j] \geq i$.

We apply these results from Line 2 through Line 6 to compute the number c of conjugates of R smaller than bR according to ω -preorder. Since $C_{\{R\}}(k)[.. \text{select}_{\$}(C_{\{R\}}(k), 1)] = C_{\{bR\}}(k + 1)[.. \text{select}_{\$}(C_{\{R\}}(k), 1)]$ for each $k \in [1.. \rho]$ by assumption on R and b ,

$$C_{\{bR\}}(\text{CA}_{\{bR\}}[i])[..x] = \begin{cases} C_{\{R\}}(\text{CA}_{\{R\}}[i])[..x] & \text{if } i \in [1..c], \\ bR & \text{if } i = c + 1, \\ C_{\{R\}}(\text{CA}_{\{R\}}[i - 1])[..x] & \text{otherwise,} \end{cases}$$

for each $i \in [1.. \rho + 1]$ with $x = \text{select}_{\$}(C_{\{bR\}}(\text{CA}_{\{bR\}}[i]), 1)$. Thus, c is also the number of conjugates of bR smaller than bR according to ω -preorder. Subsequently, Algorithm 2 updates the ceBWT index of $\{R\}$ from Line 8 through Line 17. By assumption on b and R , $\text{LCP}_{\{R\}}^{\infty}[..c] = \text{LCP}_{\{bR\}}^{\infty}[..c]$ and $\text{LCP}_{\{R\}}^{\infty}[c + 2..] = \text{LCP}_{\{bR\}}^{\infty}[c + 3..]$. From Line 8 through Line 14 we leverage Lemma 3.13 and Lemma 4.1 to compute $\text{LCP}_{\{bR\}}^{\infty}[c + 1]$ and, if $c < \rho$, $\text{LCP}_{\{bR\}}^{\infty}[c + 2]$, and update $\text{LCP}_{\{R\}}^{\infty}$. By assumption on R , $\pi(C_{\{R\}}(j)) = \pi(C_{\{bR\}}(j + 1))$ for each $j \in [1.. \rho]$. Consequently, $F_{\{R\}}[..c] = F_{\{bR\}}[..c]$ and $F_{\{R\}}[c + 1..] = F_{\{bR\}}[c + 2..]$.

Moreover, if we set $L_{\{R\}}[CA_{\{R\}}^{-1}[1]] = \pi(bR)$, then $L_{\{R\}}[..c] = L_{\{bR\}}[..c]$ and $L_{\{R\}}[c+1..] = L_{\{bR\}}[c+2..]$. It remains to compute $F_{\{bR\}}[c+1]$ and $L_{\{bR\}}[c+1]$, which are $\pi(bR)$ and $\$$, respectively. The update of both $F_{\{R\}}$ and $L_{\{R\}}$ is done from Line 15 through Line 17. ◀

► **Corollary 4.3.** *Let $R \in \Sigma_{\$}^{+}$, $\rho = |R|$, $R[\rho] = \$$, and $\text{rank}_{\$}(R, \rho) = 1$. The ceBWT index of $\{R\}$ can be constructed in $O(\rho \lg \sigma)$ bits of space and $O(\rho \frac{\lg \sigma \lg \rho}{\lg \lg \rho})$ time.*

Proof. We show the case where $\rho \geq 2$. We represent $F_{\{R[\rho-1..]\}} = \$0$, $L_{\{R[\rho-1..]\}} = 0\$$ and $LCP_{\{R[\rho-1..]\}}^{\infty} = 00$ by the data structure of Lemma 2.1. Initially, $CA_{\{R[\rho-1..]\}}^{-1}[1] = 2$. We preprocess R with Lemma 3.15 in $O(\rho)$ time to have access to $\pi(R[i..]) = \pi(\text{Rot}(R, i-1))$ for each $i \in [1..\rho]$ in $O(1)$ time. For each $i \in [1..\rho-2]$ in descending order, we apply Algorithm 2 to compute $CA_{\{R[i..]\}}^{-1}[1]$ and extend the ceBWT index of $\{R[i+1..]\}$ to that of $\{R[i..]\}$. Since the extension of the ceBWT index of $\{R[i+1..]\}$ to that of $\{R[i..]\}$ takes $O(\pi(R[i..]))$ queries for each $i \in [1..\rho-2]$, the construction of the ceBWT index of $\{R[1..]\} = \{R\}$ takes a total of $O(\rho)$ queries by Lemma 3.7. The claimed complexities then follow by Lemma 2.1. ◀

► **Remark 4.4.** Due to Remark 3.11, the previous statement yields the integer-based representation of Kim and Cho's index [20, Section 4] by substituting $\$$ with -1 in both $F_{\mathcal{T}}$ and $L_{\mathcal{T}}$. Thus, we can construct their compact index [20, Section 5] directly [20, Section A.4] while retaining the complexities as stated in Corollary 4.3.

► **Corollary 4.5.** *Let $T \in \Sigma^{+}$ and $|T| = n$. Then the ceBWT index of $\{T\}$ can be constructed in $O(n \lg \sigma)$ bits of space and $O(n \frac{\lg \sigma \lg n}{\lg \lg n})$ time.*

Proof. Let $R = T^4\$$. We construct the ceBWT index of $\{R\}$ within the claimed complexities by Corollary 4.3. During construction, we build a bit string Y of length $4n+1$ such that $Y[i] = 1$ if and only if $CA_{\{R\}}[i] \in [2..n+1]$. By choice of R , $C_{\{T\}}(i)^{\omega}[..3n] = C_{\{R\}}(i)[..3n]$ for each $i \in [1..n]$, and $C_{\{T\}}(1)^{\omega}[..3n] = C_{\{R\}}(n+1)[..3n]$. The following statements hold for each $i \in [1..n]$ by choice of Y and Lemma 3.3.

- $C_{\{T\}}(CA_{\{T\}}[i])^{\omega}[..3n] = C_{\{R\}}(CA_{\{R\}}[\text{select}_1(Y, i)])[..3n]$.
- $\pi(C_{\{T\}}(CA_{\{T\}}[i])) = \pi(C_{\{R\}}(CA_{\{R\}}[\text{select}_1(Y, i)]))$.
- $\pi(C_{\{T\}}(CA_{\{T\}}[\text{LF}_{\{T\}}[i]])) = \pi(C_{\{R\}}(CA_{\{R\}}[\text{LF}_{\{R\}}[\text{select}_1(Y, i)]))$.

Thus, $F_{\{T\}}[i] = F_{\{R\}}[\text{select}_1(Y, i)]$ and $L_{\{T\}}[i] = L_{\{R\}}[\text{select}_1(Y, i)]$ for each $i \in [1..n]$. Moreover, $LCP_{\{T\}}^{\infty}[i] = \text{RNV}_{LCP_{\{R\}}^{\infty}}(\text{select}_1(Y, i-1) + 1, \text{select}_1(Y, i), -1)$ for each $i \in [2..n]$ due to Lemma 3.13, and $LCP_{\{T\}}^{\infty}[1] = 0$. Hence, we can transform the ceBWT index of $\{R\}$ to index $\{T\}$ with $O(n)$ queries and operations. The claimed complexities now follow by Lemma 2.1. ◀

4.2 Extending an Existing ceBWT Index

Let $\emptyset \neq \mathcal{R} \subset \Sigma^{+}$ be a non-empty set of strings and $\rho = |CA_{\mathcal{R}}|$ denote the accumulated length of all strings in $\mathcal{R}$. In this subsection, we assume we have the ceBWT index of $\mathcal{R}$ at hand, and want to extend it by another text $S \in \Sigma^{+}$ of length λ to index $\mathcal{S} = \mathcal{R} \cup \{S\}$. To achieve this, we compute the integers $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ for each $i \in [1..\lambda]$, whose definitions follow. Let $\text{cnt}_{\mathcal{R}}(V) = |\{i \in [1..\rho] \mid C_{\mathcal{R}}(CA_{\mathcal{R}}[i]) \preceq_{\omega} V\}|$,

$$\begin{aligned} \text{plcp}_{\mathcal{R}}^{\infty}(V) &= \begin{cases} -1 & \text{if } \text{cnt}_{\mathcal{R}}(V) = 0, \\ \text{lcp}^{\infty}(C_{\mathcal{R}}(CA_{\mathcal{R}}[\text{cnt}_{\mathcal{R}}(V)]), V) & \text{otherwise, and} \end{cases} \\ \text{slcp}_{\mathcal{R}}^{\infty}(V) &= \begin{cases} \text{lcp}^{\infty}(C_{\mathcal{R}}(CA_{\mathcal{R}}[\text{cnt}_{\mathcal{R}}(V) + 1]), V) & \text{if } \text{cnt}_{\mathcal{R}}(V) < \rho, \\ -1 & \text{otherwise,} \end{cases} \end{aligned}$$

for each $V \in \Sigma_{\mathcal{R}}^+$. We can apply the techniques exhibited in Lemma 4.2 to compute the helper values.

■ **Algorithm 3** Computing $c = \text{cnt}_{\mathcal{R}}(V)$, $p = \text{plcp}_{\mathcal{R}}^{\infty}(V)$ and $s = \text{slcp}_{\mathcal{R}}^{\infty}(V)$. Here, $\emptyset \neq \mathcal{R} \subset \Sigma^+$, $\rho = |\text{CA}_{\mathcal{R}}|$, $V \in \Sigma_{\mathcal{R}}^+$, $\text{rank}_{\infty}(V, |V|) \geq 1$, and $y = \text{cnt}_{\mathcal{R}}(\text{Rot}(V, 1))$.

```

1 Function next( $\pi(V), y, \text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)), \text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)), F_{\mathcal{R}}, L_{\mathcal{R}}, \text{LCP}_{\mathcal{R}}^{\infty}$ ):
2   if  $\pi(V) = \$$  then return 0, -1, 0 ;
3   if  $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)) \geq \pi(V) + 1$  then  $[\ell..r] \leftarrow \text{MI}_{\text{LCP}_{\mathcal{R}}^{\infty}}(y + 1, \pi(V) + 1)$  ;
4   else if  $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)) \geq \pi(V) + 1$  then  $[\ell..r] \leftarrow \text{MI}_{\text{LCP}_{\mathcal{R}}^{\infty}}(y, \pi(V) + 1)$  ;
5   else  $[\ell..r] \leftarrow [y + 1..y]$  ;
6    $c \leftarrow \text{rnkent}_{L_{\mathcal{R}}}(y + 1, r, \pi(V) + 1, \sigma) + \text{rnkent}_{L_{\mathcal{R}}}(\ell, y, \pi(V), \sigma)$ ;
7   for  $i \leftarrow \min\{\pi(V), \text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1))\}$  to 0 do
8      $r' \leftarrow \ell - 1$ ;
9      $[\ell..r] \leftarrow \text{MI}_{\text{LCP}_{\mathcal{R}}^{\infty}}(y, i)$ ;
10     $c \leftarrow c + \text{rnkent}_{L_{\mathcal{R}}}(\ell, r', i, \sigma)$ ;
11  if  $c = 0$  then  $p \leftarrow -1$  ;
12  else if  $\text{FL}_{\mathcal{R}}[c] \leq y$  and  $F_{\mathcal{R}}[c] = \pi(V) < \text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1))$  then
13    if  $\text{FL}_{\mathcal{R}}[c] = y$  then  $p \leftarrow \text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)) - \pi(V) + 1$  ;
14    else if  $\pi(V) \geq \text{RNV}_{\text{LCP}_{\mathcal{R}}^{\infty}}(\text{FL}_{\mathcal{R}}[c] + 1, y, -1)$  then  $p \leftarrow 1$  ;
15    else  $p \leftarrow \min\{\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)), \text{RNV}_{\text{LCP}_{\mathcal{R}}^{\infty}}(\text{FL}_{\mathcal{R}}[c] + 1, y, -1)\} - \pi(V) + 1$  ;
16  else  $p \leftarrow 1$  ;
17  if  $c = \rho$  then  $s \leftarrow -1$  ;
18  else if  $\text{FL}_{\mathcal{R}}[c + 1] \geq y + 1$  and  $F_{\mathcal{R}}[c + 1] = \pi(V) < \text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1))$  then
19    if  $\text{FL}_{\mathcal{R}}[c + 1] = y + 1$  then  $s \leftarrow \text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)) - \pi(V) + 1$  ;
20    else if  $\pi(V) < \text{RNV}_{\text{LCP}_{\mathcal{R}}^{\infty}}(y + 2, \text{FL}_{\mathcal{R}}[c + 1], -1)$  then
21       $s \leftarrow \min\{\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1)), \text{RNV}_{\text{LCP}_{\mathcal{R}}^{\infty}}(y + 2, \text{FL}_{\mathcal{R}}[c + 1], -1)\} - \pi(V) + 1$ ;
22    else  $s \leftarrow 1$  ;
23  else  $s \leftarrow 1$  ;
24  return  $c, p, s$ 

```

► **Lemma 4.6.** Let $\emptyset \neq \mathcal{R} \subset \Sigma^+$, $\rho = |\text{CA}_{\mathcal{R}}|$, and $V \in \Sigma_{\mathcal{R}}^+$ with $\text{rank}_{\infty}(V, |V|) \geq 1$. Then Algorithm 3 correctly computes $\text{cnt}_{\mathcal{R}}(V)$, $\text{plcp}_{\mathcal{R}}^{\infty}(V)$ and $\text{slcp}_{\mathcal{R}}^{\infty}(V)$.

Proof. Algorithm 3 takes $\pi(V)$, $y = \text{cnt}_{\mathcal{R}}(\text{Rot}(V, 1))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1))$, $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1))$, and the ceBWT index of $\mathcal{R}$ as input. If $\pi(V) = \$$, then we return $\text{cnt}_{\mathcal{R}}(V) = 0$, $\text{plcp}_{\mathcal{R}}^{\infty}(V) = -1$ and $\text{slcp}_{\mathcal{R}}^{\infty}(V) = 0$ in Line 2 since $\mathcal{R} \subset \Sigma^+$. Thus, assume $\pi(V) > \$$. For each $i \in [0.. \pi(V)]$, let $J_i = [\ell_i..r_i]$ maximal such that $r_i \leq y$ and $\text{lcp}^{\infty}(V, C_{\mathcal{R}}(\text{CA}_{\mathcal{R}}[j])) = i$ for each $j \in J_i$. The following statements are due to Lemma 3.8 and $\mathcal{R} \subset \Sigma^+$.

- If $j \in [y + 1.. \rho]$, then $C_{\mathcal{R}}(\text{CA}_{\mathcal{T}}[\text{LF}_{\mathcal{R}}[j]]) \prec_{\omega} V$ if and only if $L_{\mathcal{R}}[j] \geq \pi(V) + 1$ and $\text{lcp}^{\infty}(C_{\mathcal{R}}(\text{CA}_{\mathcal{T}}[i]), \text{Rot}(V, j)) \geq \pi(V) + 1$.
- If $j \in [r_{\pi(V)} + 1..y]$, then $C_{\mathcal{R}}(\text{CA}_{\mathcal{T}}[\text{LF}_{\mathcal{R}}[j]]) \prec_{\omega} V$ if and only if $L_{\mathcal{R}}[j] \geq \pi(V)$.
- If $i \in [0.. \pi(V)]$ and $j \in J_i$, then $C_{\mathcal{R}}(\text{CA}_{\mathcal{T}}[\text{LF}_{\mathcal{R}}[j]]) \prec_{\omega} V$ if and only if $L_{\mathcal{T}}[j] \geq i$.

We apply these results from Line 3 through Line 10 to compute $\text{cnt}_{\mathcal{R}}(V)$. Leveraging Lemma 4.1 and (the proof of) Lemma 3.13, we then compute $\text{plcp}_{\mathcal{R}}^{\infty}(V)$ and $\text{slcp}_{\mathcal{R}}^{\infty}(V)$ from Line 11 through Line 16 and from Line 17 through Line 23, respectively. ◀

► **Lemma 4.7.** *Let $\emptyset \neq \mathcal{R} \subset \Sigma^+$, $\rho = |\text{CA}_{\mathcal{R}}|$, $V \in \Sigma_{\$}^+$, and $\text{rank}_x(V, |V|) \geq 1$. If $\pi(V) \neq \$$, then $\text{cnt}_{\mathcal{R}}(V)$, $\text{plcp}_{\mathcal{R}}^{\infty}(V)$ and $\text{slcp}_{\mathcal{R}}^{\infty}(V)$ can be computed from $\pi(V)$, $\text{cnt}_{\mathcal{R}}(\text{Rot}(V, 1))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1))$ and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 1))$ in $O((1 + \pi(V))^{\frac{\lg \sigma \lg \rho}{\lg \lg \rho}})$ time with the ceBWT index of $\mathcal{R}$.*

Proof. We apply Algorithm 3. Correctness follows from Lemma 4.6, and the time complexity is due to the loop of Algorithm 3 in Line 7 and Lemma 2.1. ◀

For the iterative construction, we augment the ceBWT index by a dynamic bit string E with the invariant that E is zeroed before and after each extension with a new string. We use E to temporarily mark modified parts in the ceBWT such that we retain the functionality of the index even during construction. The length of E is the number ρ of characters indexed by the ceBWT index, which we call in the next lemma the *augmented ceBWT index* for clarity.

► **Lemma 4.8.** *Let $\emptyset \neq \mathcal{R} \subset \Sigma^+$, $\rho = |\text{CA}_{\mathcal{R}}|$, $S \in \Sigma^+$, $\mathcal{S} = \mathcal{R} \cup \{S\}$, $\lambda = |S|$, and $z = \max\{|V| \mid V \in \mathcal{S}\}$. The augmented ceBWT index of $\mathcal{R}$ can be extended to index $\mathcal{S}$ in $O((\lambda + \rho) \lg \sigma)$ bits of space and $O(z^{\frac{\lg \sigma \lg(\lambda + \rho)}{\lg \lg(\lambda + \rho)}})$ time.*

Proof. We compute the ceBWT index of $\{S\}$ within the claimed complexities by Corollary 4.5. Then we compute $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ for each $i \in [1.. \lambda]$.

Case 1. Assume $\text{Rot}(S, i) =_{\omega} R$ for some $R \in \mathcal{R}$ and $i \in [1.. \lambda]$. Then $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i)) = r$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i)) = \text{rank}_x((\text{Rot}(S, i)), \lambda)$, and, if $r < \rho$, $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i)) = \text{LCP}_{\mathcal{R}}^{\infty}[r + 1]$ by Lemma 3.3, where $i \in [1.. \lambda]$ and r is the right boundary of $\text{CR}_{\mathcal{R}}(\text{Rot}(S, i)^{\omega}[\dots 3z])$. Since $\text{rank}_x((\text{Rot}(S, i - 1)), \lambda) = \text{rank}_x((S^{\omega}[i..4z]), 4z - i + 1)$ for each $i \in [2.. \lambda + 1]$, the last λ steps of the backward search for $S^{\omega}[2..4z]$ yield the necessary values to compute $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ for all $i \in [1.. \lambda]$ in descending order within the claimed complexities by Theorem 3.16.

Case 2. Assume $\text{Rot}(S, i) \neq_{\omega} R$ for each $R \in \mathcal{R}$ and $i \in [1.. \lambda]$. Let $V = S^{\omega}[\dots 4z] \cdot \$$. By assumption and Corollary 3.5, $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i)) = \text{cnt}_{\mathcal{R}}(\text{Rot}(V, i))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i)) = \text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, i))$, and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i)) = \text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, i))$ for each $i \in [1.. \lambda]$. We leverage Lemma 3.15 to preprocess V within the claimed complexities. Since $\text{cnt}_{\mathcal{R}}(\text{Rot}(V, 4z)) = 0$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 4z)) = -1$ and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, 4z)) = 0$ by construction, we can now use Lemma 4.7 to compute $\text{cnt}_{\mathcal{R}}(\text{Rot}(V, j))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, j))$ and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(V, j))$ for each $j \in [1.. 4z - 1]$ in descending order. Hence, we obtain $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i))$, $\text{plcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ and $\text{slcp}_{\mathcal{R}}^{\infty}(\text{Rot}(S, i))$ for all $i \in [1.. \lambda]$ in descending order within the claimed complexities by Lemma 3.7.

Leveraging the backward search on the ceBWT index of $\{S\}$, we store the $\text{plcp}_{\mathcal{R}}^{\infty}$ -values and $\text{slcp}_{\mathcal{R}}^{\infty}$ -values in lex-order, which takes $O(\lambda \lg \sigma)$ bits of space. To store the $\text{cnt}_{\mathcal{R}}$ -values in lex-order and stay within the claimed time and space complexity, we utilize E , which we assume to be represented by the data structure of Lemma 2.1. For each $i \in [1.. \lambda]$ in descending order, we call $\text{insert}_E(\text{select}_0(E, \text{cnt}_{\mathcal{R}}(\text{Rot}(S, i))) + 1, 1)$, and discard $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i))$ once $\text{cnt}_{\mathcal{R}}(\text{Rot}(S, i - 1))$ has been computed. Then $\text{cnt}_{\mathcal{R}}(C_{\{S\}}(\text{CA}_{\{S\}}[i])) = \text{rank}_0(E, \text{select}_1(E, i))$. The following statements for each $i \in [1.. \lambda + \rho]$ are due to construction.

- If $E[i] = 0$, then $F_S[i] = F_{\mathcal{R}}[\text{rank}_0(E, i)]$ and $L_S[i] = L_{\mathcal{R}}[\text{rank}_0(E, i)]$.
- If $E[i] = 1$, then $F_S[i] = F_{\{S\}}[\text{rank}_1(E, i)]$ and $L_S[i] = L_{\{S\}}[\text{rank}_1(E, i)]$.
- $\text{LCP}_S^{\infty}[1] = 0$.
- If $i \geq 2$, $E[i - 1] = 1$ and $E[i] = 1$, then $\text{LCP}_S^{\infty}[i] = \text{LCP}_{\{S\}}^{\infty}[\text{rank}_1(E, i)]$.
- If $i \geq 2$, $E[i - 1] = 1$ and $E[i] = 0$, then $\text{LCP}_S^{\infty}[i] = \text{slcp}_{\mathcal{R}}^{\infty}(C_{\{S\}}(\text{CA}_{\{S\}}[\text{rank}_1(E, i)]))$.
- If $i \geq 2$, $E[i - 1] = 0$ and $E[i] = 1$, then $\text{LCP}_S^{\infty}[i] = \text{plcp}_{\mathcal{R}}^{\infty}(C_{\{S\}}(\text{CA}_{\{S\}}[\text{rank}_1(E, i)]))$.

■ If $i \geq 2$, $E[i-1] = 0$ and $E[i] = 0$, then $\text{LCP}_S^\infty[i] = \text{LCP}_R^\infty[\text{rank}_0(E, i)]$.

Consequently, $O(\lambda)$ queries and operations are needed to update the ceBWT index of R to index S . Finally, we zero E . The claimed complexities then follow from Lemma 2.1. ◀

We are finally able to state the main result of this section.

► **Theorem 4.9.** *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma^+$ and $n = |T_1 \cdots T_d|$. The ceBWT index of $\mathcal{T}$ can be constructed in $O(n \lg \sigma)$ bits of space and $O(n \frac{\lg \sigma \lg n}{\lg \lg n})$ time.*

Proof. Let $|T_{k-1}| \leq |T_k|$ for each $k \in [2..d]$, and let E_j denote a zeroed bit string of length $|T_1 \cdots T_j|$ for each $j \in [1..d]$. First, we construct E_1 and the ceBWT index of $\{T_1\}$ within the claimed complexities by Lemma 4.5, where each string is represented by the data structure of Lemma 2.1. Subsequently, we iteratively extend the ceBWT index of $\{T_1, \dots, T_{k-1}\}$ augmented by E_{k-1} to the ceBWT index of $\{T_1, \dots, T_k\}$ augmented by E_k for each $k \in [2..d]$ in ascending order leveraging Lemma 4.8. Then the ceBWT index of $\mathcal{T}$ is constructed in $O((n + |T_d|) \lg \sigma) = O(n \lg \sigma)$ bits of space and $O((\sum_{k=1}^d |T_k|) \frac{\lg \sigma \lg n}{\lg \lg n}) = O(n \frac{\lg \sigma \lg n}{\lg \lg n})$ time. ◀

4.3 Showcase ceBWT Index Extension

We extend the ceBWT index of our running example $\mathcal{T}$ by $S = 73152$ to index $\mathcal{S} = \mathcal{T} \cup \{S\}$. We have $z = \max\{|V| \mid V \in \mathcal{S}\} = |S| = 5$ and $\text{CR}_{\mathcal{T}}(S^3) = \emptyset$. For the extension, we follow the proof of Lemma 4.8. The intermediate and final results are shown in Figure 7.

4.4 Removal of Texts from the ceBWT Index

In addition to an incremental addition of texts to an existing ceBWT index, the index supports the removal of texts.

► **Lemma 4.10.** *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma^+$ and $n = |T_1 \cdots T_d|$. Then T_k can be removed from the augmented ceBWT index of $\mathcal{T}$ in $O(|T_k| \frac{\lg \sigma \lg n}{\lg \lg n})$ time for each $k \in [1..d]$.*

Proof. Fix some $k \in [1..d]$. We perform the backward search for T_k^4 . We mark the left boundary ℓ of $\text{CR}_{\mathcal{T}}(T_k^4)$ in E with a '1' and then consecutively compute $\text{LF}_{\mathcal{T}}[\ell], \text{LF}_{\mathcal{T}}[\text{LF}_{\mathcal{T}}[\ell]], \dots$ while marking each position returned in E until we reach a position that has already been marked. Then either we performed the LF-mapping T_k^4 times or $\langle T_k \rangle_r$ is not primitive. For the latter case we simply track the number c of computed LF-mappings not mapping to a marked position and mark the $\frac{|T_k|}{c} - 1$ positions following each of the initially marked positions in E . Leveraging Lemma 3.13 and E , the update of the augmented ceBWT index of $\mathcal{T}$ to that of $\mathcal{T} - \{T_k\}$ is now straightforward with the claimed complexities following from Lemma 2.1 and correctness by Lemma 3.3. ◀

5 $O(n)$ -bit Index

The compact representation of the ceBWT index for alphabets of arbitrary size is almost a one-to-one adaptation of Kim and Cho's [20, Section 5] results. Consequently, we keep it brief. For any $V \in \Sigma^+$ and $c \in \Sigma$, let $B_V^{(c)}$ denote a bit string such that $|B_V^{(c)}| = \text{rnkcnt}_V(1, |V|, c, \sigma)$, $\text{rank}_1(B_V^{(c)}, |B_V^{(c)}|) = \text{rnkcnt}_V(1, |V|, c+1, \sigma)$, and $B_V^{(c)}[i] = 0$ if and only if the i -th symbol not smaller than c in V equals c , i.e., $B_V^{(c)}$ has the length of the longest subsequence of V consisting of characters not smaller than c and its entries mark the positions of c in this

i	$\pi(\text{Rot}(V, i))$	$\text{cnt}_{\mathcal{T}}(\text{Rot}(V, i))$	$\text{plcp}_{\mathcal{T}}^{\infty}(\text{Rot}(V, i))$	$\text{slcp}_{\mathcal{T}}^{\infty}(\text{Rot}(V, i))$
20	\$	0	-1	0
19	0	0	-1	1
18	0	7	1	2
17	2	3	1	1
16	0	9	2	2
15	0	11	2	-1
14	2	5	1	1
13	0	11	2	-1
12	3	5	1	1
11	0	11	2	-1
10	0	11	2	-1
9	2	5	1	1
8	0	11	2	-1
7	3	5	1	1
6	0	11	2	-1
5	0	11	2	-1
4	2	5	1	1
3	0	11	2	-1
2	3	5	1	1
1	0	11	2	-1

(a) Helper values for $V = S^4\$$.

i	$F_{\{S\}}[i]$	$L_{\{S\}}[i]$	$\text{LCP}_{\{S\}}^{\infty}[i]$	$\text{plcp}_{\mathcal{T}}^{\infty}(C_{\{S\}}(\text{CA}_{\{S\}}[i]))$	$\text{slcp}_{\mathcal{T}}^{\infty}(C_{\{S\}}(\text{CA}_{\{S\}}[i]))$
1	3	0	0	1	1
2	2	0	1	1	1
3	0	0	1	2	-1
4	0	3	2	2	-1
5	0	2	2	2	-1

(b) The ceBWT index of $\{S\}$ and helper values for S in lex-order.

i	$F_S[i]$	$L_S[i]$	$\text{LCP}_S^{\infty}[i]$	$E[i]$	i	$F_S[i]$	$L_S[i]$	$\text{LCP}_S^{\infty}[i]$	$E[i]$
1	1	0	0	0	9	1	2	2	0
2	2	1	1	0	10	0	1	1	0
3	2	0	1	0	11	0	1	2	0
4	2	0	1	0	12	0	2	2	0
5	2	0	1	0	13	0	2	2	0
6	3	0	1	1	14	0	0	2	1
7	2	0	1	1	15	0	3	2	1
8	1	2	1	0	16	0	2	2	1

(c) The augmented ceBWT index of S before zeroing E .

■ **Figure 7** Extending the ceBWT index of the running example $\mathcal{T}$ presented in Figure 5 and Figure 6 to index $\mathcal{S} = \mathcal{T} \cup \{S\}$, where $S = 73152$.

■ **Algorithm 4** Computing the conjugate range $\text{CR}_{\mathcal{T}}(P[i..])$. Here, $\emptyset \neq \mathcal{T} \subset \Sigma^+$, $P \in \Sigma^+$, $m = |P|$, $i \in [1..m]$, $[\ell..r] = \text{CR}_{\mathcal{T}}(P[i+1..])$, $h = \pi(P[i..] \cdot \$)$, $e = \text{rank}_{\infty}(\langle P[i..] \rangle, m - i + 1)$, and $c \in \Sigma$ maximal such that $B_{L_{\mathcal{T}}}^{(c)} \neq \varepsilon$.

```

1 Function altcrangeupd( $e, h, [\ell..r], B_{F_{\mathcal{T}}}^{(0)}, \dots, B_{F_{\mathcal{T}}}^{(c)}, B_{L_{\mathcal{T}}}^{(0)}, \dots, B_{L_{\mathcal{T}}}^{(c)}$ ):
2   if  $h > c$  then return  $[r + 1, r]$  ; //  $\text{CR}_{\mathcal{T}}(P[i..]) = \emptyset$ 
3   for  $j \leftarrow 0$  to  $h - 1$  do
4      $\ell \leftarrow \text{rank}_1(B_{L_{\mathcal{T}}}^{(j)}, \ell - 1) + 1$ ;
5      $r \leftarrow \text{rank}_1(B_{L_{\mathcal{T}}}^{(j)}, r)$ ;
6     if  $\ell > r$  then return  $[r + 1, r]$  ; //  $\text{CR}_{\mathcal{T}}(P[i..]) = \emptyset$ 
7   if  $e > 1$  then
8     if  $\text{rank}_0(B_{L_{\mathcal{T}}}^{(h)}, r) - \text{rank}_0(B_{L_{\mathcal{T}}}^{(h)}, \ell - 1) = 0$  then
9       return  $[r + 1, r]$  ; //  $\text{CR}_{\mathcal{T}}(P[i..]) = \emptyset$ 
10     $\ell \leftarrow \text{select}_0(B_{F_{\mathcal{T}}}^{(h)}, \text{rank}_0(B_{L_{\mathcal{T}}}^{(h)}, \ell - 1) + 1)$ ;
11     $r \leftarrow \text{select}_0(B_{F_{\mathcal{T}}}^{(h)}, \text{rank}_0(B_{L_{\mathcal{T}}}^{(h)}, r))$ ;
12  for  $j \leftarrow h$  to  $1$  do
13     $\ell \leftarrow \text{select}_1(B_{F_{\mathcal{T}}}^{(j-1)}, \ell)$ ;
14     $r \leftarrow \text{select}_1(B_{F_{\mathcal{T}}}^{(j-1)}, r)$ ;
15  return  $[\ell..r]$ ;

```

subsequence with a '0'. For example, $B_{L_{\mathcal{T}}}^{(1)} = B_{01000221122}^{(1)} = 0110011$ and $B_{L_{\mathcal{T}}}^{(0)} = 01000111111$. The following statement is a consequence of Lemma 3.7.

► **Lemma 5.1** ([20, Section 5.3]). *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma^+$ and $c \in \Sigma$ maximal such that $B_{L_{\mathcal{T}}}^{(c)} \neq \varepsilon$. Then $B_{F_{\mathcal{T}}}^{(0)}, \dots, B_{F_{\mathcal{T}}}^{(c)}, B_{L_{\mathcal{T}}}^{(0)}, \dots, B_{L_{\mathcal{T}}}^{(c)}$ can be stored in $3n + o(n)$ bits of space while supporting rank and select queries in $O(1)$ time.*

The data structures in the setting of Lemma 5.1 suffice to compute the conjugate range update. We need a technical result which is an immediate consequence of Lemma 3.8.

► **Corollary 5.2** ([20, Lemma 9]). *Let $\emptyset \neq \mathcal{T} \subset \Sigma^+$, $P \in \Sigma^+$, $e = \text{rank}_x(\langle P \rangle, |P|)$ and $[\ell..r] = \text{CR}_{\mathcal{T}}(P)$. For each $i, j \in [1..|P|]$, if $\min\{L_{\mathcal{T}}[i], L_{\mathcal{T}}[j]\} \geq e$ and either $i < \ell \leq j$ or $i \leq r < j$, then $\text{LF}_{\mathcal{T}}[i] < \text{LF}_{\mathcal{T}}[j]$.*

► **Lemma 5.3** ([20, Section 6]). *Let $\emptyset \neq \mathcal{T} \subset \Sigma^+$, $P \in \Sigma^+$, $m = |P|$, $i \in [1..m]$, $[\ell..r] = \text{CR}_{\mathcal{T}}(P[i+1..])$, $h = \pi(P[i..] \cdot \$)$, and $e = \text{rank}_x(\langle P[i..] \rangle, m - i + 1)$. Then Algorithm 4 correctly computes $[\ell'..r'] = \text{CR}_{\mathcal{T}}(P[i..])$.*

Proof. From Line 2 through Line 6, we determine the substring $B_{L_{\mathcal{T}}}^{(h)}[\ell''..r'']$ of $B_{L_{\mathcal{T}}}^{(h)}$ which corresponds to the longest subsequence of $L_{\mathcal{T}}[\ell..r]$ such that each entry is not smaller than h . If $B_{L_{\mathcal{T}}}^{(h)}[\ell''..r''] = \varepsilon$, we return an empty interval in Line 2 or Line 6 for the conjugate range $\text{CR}_{\mathcal{T}}(P[i..])$ with correctness following from Lemma 3.12. Hence, assume $B_{L_{\mathcal{T}}}^{(h)}[\ell''..r''] \neq \varepsilon$. We consider two cases.

Case 1 Assume $e = 1$. By Lemma 3.12, $\text{LF}_{\mathcal{T}}[j] \in \text{CR}_{\mathcal{T}}(P[i..])$ if and only if $L_{\mathcal{T}}[j] \geq h$ for each $j \in [\ell..r]$, i.e., $|\text{CR}_{\mathcal{T}}(P[i..])| = |B_{L_{\mathcal{T}}}^{(h)}[\ell''..r'']|$. Moreover, the length of the longest subsequence of $F_{\mathcal{T}}[1..\ell' - 1]$ such that each entry is not smaller than h is exactly $\ell'' - 1$ by Corollary 5.2. Thus, the ℓ'' -th and r'' -th entries of $F_{\mathcal{T}}$ evaluating to at least h are ℓ' and r' , respectively, with the latter values computed from Line 12 through Line 14.

i	1	2	3	4	5	6	7	8
$P[i]$	6	3	7	2	4	8	7	1
$M[i].q$	3	5	4	3	3	2	2	1
$M[i].\ell$	8	6	8	1	4	8	8	1
$M[i].r$	11	7	9	3	5	11	11	11

■ **Figure 8** The circular matching statistics M between $P = 63724871$ and our running example $\mathcal{T} = \{512, 5363, 4478\}$.

Case 2 Assume $e > 1$. By Lemma 3.12, $\text{LF}_{\mathcal{T}}[j] \in \text{CR}_{\mathcal{T}}(P[i..])$ if and only if $\text{L}_{\mathcal{T}}[j] = h$ for each $j \in [\ell..r]$, i.e., $|\text{CR}_{\mathcal{T}}(P[i..])| = \text{rank}_0(\text{B}_{\text{L}_{\mathcal{T}}}^{(h)}[\ell''..r''], r'' - \ell'' + 1)$. We catch the case where $|\text{CR}_{\mathcal{T}}(P[i..])|$ is empty in Line 9. Thus, assume $|\text{CR}_{\mathcal{T}}(P[i..])|$ is nonempty. By Corollary 3.9, the $\text{select}_0(\text{B}_{\text{F}_{\mathcal{T}}}^{(h)}, \text{rank}_0(\text{B}_{\text{L}_{\mathcal{T}}}^{(h)}, \ell'' - 1) + 1)$ -th and the $\text{select}_0(\text{B}_{\text{F}_{\mathcal{T}}}^{(h)}, \text{rank}_0(\text{B}_{\text{L}_{\mathcal{T}}}^{(h)}, r''))$ -th h in $\text{F}_{\mathcal{T}}$ are at position ℓ' and r' , respectively, with the latter values computed from Line 10 through Line 14. ◀

► **Theorem 5.4.** Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma^+$ and $n = |T_1 \cdots T_d|$. There exists a data structure occupying $3n + o(n)$ bits of space that solves COUNT in $O(m)$ time, where m is the pattern length. With additional $O(n \lg \sigma)$ bits of space, the index can be constructed in $O(n \frac{\lg \sigma \lg n}{\lg \lg n})$ time.

Proof. The space complexity follows from Lemma 5.1. For the backward search, we preprocess a pattern $P \in \Sigma^+$ of length m with Lemma 3.15 and compute $\text{CR}_{\mathcal{T}}(P[i..])$ from $\text{CR}_{\mathcal{T}}(P[i+1..])$ for each $i \in [1..m]$ in descending order leveraging Lemma 5.3. Since the conjugate range update with Algorithm 4 takes $O(\pi(P[i..] \cdot \$))$ time for each $i \in [1..m]$, the time complexity for the backward search follows by Lemma 3.7. The claimed complexities for the construction follow from Theorem 4.9 and Fischer [9, Theorem 1]. ◀

6 Circular Matching Statistics

We give a possible application for the ceBWT index by generalizing Chang and Lawler's [3] matching statistics. The *circular matching statistics* between $\mathcal{T} \subset \Sigma^+$ and $P \in \Sigma^+$ of length m is an array $M[1..m]$, whose entries are triples of a prefix length $M[i].q \in [0..m]$, a left interval boundary $M[i].\ell \in [1..n]$ and a right interval boundary $M[i].r \in [1..n]$ such that for each $i \in [1..m]$

- $M[i].q = \max\{\text{lcp}(\langle C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[j])^{\omega}[\dots m] \rangle, \langle P[i..] \rangle) \mid j \in [1..n]\},$
- $M[i].\ell \leq M[i].r,$
- $j \in [M[i].\ell..M[i].r] \Leftrightarrow \text{lcp}(\langle C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[j])^{\omega}[\dots m] \rangle, \langle P[i..] \rangle) = M[i].q.$

An example can be found in Figure 8. We adapt Ohlebusch and colleagues' [28] results and compute the circular matching statistics leveraging the backward search with the ceBWT index. The main idea is the computation of sufficiently many conjugate ranges of $P[i..j]$ where $i, j \in [1..m]$. We start with $i = j = m$ and decrease i until $\text{CR}_{\mathcal{T}}(P[i..j])$ is empty, and then decrease j until $\text{CR}_{\mathcal{T}}(P[i..j])$ is non-empty. The latter operation is called a *contraction*. We repeat until $i = 1$ and either $j = 0$ or $\text{CR}_{\mathcal{T}}(P[1..j])$ is non-empty. To allow for an efficient contraction, we utilize a modified LCP array. Let $n' = \max\{n_k \mid k \in [1..d]\}$ and denote by $\text{LCP}_{\mathcal{T}}^{\circ}$ a string of length n over $[0..4n']$ such that $\text{LCP}_{\mathcal{T}}^{\circ}[1] = 0$ and

$$\text{LCP}_{\mathcal{T}}^{\circ}[i] = \text{lcp}(\langle C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i-1])^{\omega}[\dots 4n'] \rangle, \langle C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i])^{\omega}[\dots 4n'] \rangle)$$

■ **Algorithm 5** Computing the circular matching statistics between $\mathcal{T} \subset \Sigma^+$ and $P \in \Sigma^+$. Here, $c \in \Sigma$ maximal such that $B_{L_{\mathcal{T}}}^{(c)} \neq \varepsilon$.

```

1 Function cms( $P, F_{\mathcal{T}}, L_{\mathcal{T}}, LCP_{\mathcal{T}}^{\circ}, LCP_{\mathcal{T}}^{\circ}$ ):
2   initialize  $M$ ;
3    $(q, \ell, r, i) \leftarrow (0, 1, n, |P|)$ ;
4   while  $i \geq 1$  do
5      $[\ell'..r'] \leftarrow \text{altcrangeupd}(\text{rank}_x(\langle P[i..] \rangle, q+1), \pi(P[i..i+q] \cdot \$), [\ell..r],$ 
6        $B_{F_{\mathcal{T}}}^{(0)}, \dots, B_{F_{\mathcal{T}}}^{(c)}, B_{L_{\mathcal{T}}}^{(0)}, \dots, B_{L_{\mathcal{T}}}^{(c)})$ ; // Alg. 4
7     if  $\ell' \leq r'$  then // expand previous match
8        $q \leftarrow q + 1$ ;
9        $M[i] \leftarrow (q, \ell', r')$ ;
10       $[\ell..r] \leftarrow [\ell'..r']$ ;
11       $i \leftarrow i - 1$ ;
12    else if  $q = 0$  then // reset
13       $M[i] \leftarrow (0, 1, n)$ ;
14       $i \leftarrow i - 1$ ;
15    else // contract previous match
16       $q \leftarrow q - 1$ ;
17       $[\ell..r] \leftarrow \text{MI}_{LCP_{\mathcal{T}}^{\circ}}(r, q)$ ;
18  return  $M$ ;

```

for each $i \in [2..n]$. Note that by Lemma 3.3 and for each $i \in [2..n]$, $C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i-1]) =_{\omega} C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i])$ if and only if $LCP_{\mathcal{T}}^{\circ}[i] = 4n'$. The following statement is a consequence of Ohlebusch et al. [28, Section 3].

► **Lemma 6.1.** *Algorithm 5 correctly computes the circular matching statistics between $\mathcal{T} \subset \Sigma^+$ and $P \in \Sigma^+$.*

► **Corollary 6.2.** *There exists a data structure occupying $n \lg 4n' + 5n + o(n)$ bits of space that supports the computation of circular matching statistics between $\mathcal{T} \subset \Sigma^+$ and any pattern $P \in \Sigma^+$ in $O(m \lg n)$ time, where $m = |P|$ and n' is the maximum length of all texts in $\mathcal{T}$.*

Proof. The data structure is the ceBWT index augmented by $LCP_{\mathcal{T}}^{\circ}$, which we represent by both a simple string and by Fischer and colleagues' [9] $2n + o(n)$ -bit data structure answering the query $\text{RmQ}_{LCP_{\mathcal{T}}^{\circ}}(i, j) = \text{RNV}_{LCP_{\mathcal{T}}^{\circ}}(i, j, -1)$ in $O(1)$ time for each $i, j \in [1..n]$ with $i \leq j$, which results in the claimed space complexity.

We examine Algorithm 5. To obtain the variables passed in Line 5 we utilize Lemma 3.15 and the parent distance encoding of $P \cdot \$$, which can be extracted as a side product of the preprocessing step. Then the computation of Line 5 takes $O(m)$ time across all iterations of the while-loop by (the proof of) Theorem 5.4.

Moreover, we can perform the MI query of Line 16 by exponential search on the RmQ values of ranges in $LCP_{\mathcal{T}}^{\circ}$, i.e., Line 16 is computed in $O(\lg n)$ time. As each of the cases in the while-loop is entered at most m times until termination, the claim follows from Lemma 6.1. ◀

6.1 Constructing $\text{LCP}_{\mathcal{T}}^{\circ}$

We can construct $\text{LCP}_{\mathcal{T}}^{\circ}$ similarly to $\text{LCP}_{\mathcal{T}}^{\infty}$ and alongside the ceBWT index. For this, three technical but necessary results follow, the first two of which are adaptations of Lemma 4.1 and Lemma 3.13, respectively. Both adaptations are proven similarly to their respective counterpart.

► **Lemma 6.3.** *Let $V, U \in \Sigma_{\mathbb{S}}^+$, $\max\{|U|, |V|\} \leq \lambda$, $e = \text{lcp}^{\infty}(\text{Rot}(V, 1), \text{Rot}(U, 1))$ and $s = \text{lcp}(\langle \text{Rot}(V, 1)^{\omega}[\dots 4\lambda] \rangle, \langle \text{Rot}(U, 1)^{\omega}[\dots 4\lambda] \rangle)$. Then*

$$\text{lcp}(\langle V^{\omega}[\dots 4\lambda] \rangle, \langle U^{\omega}[\dots 4\lambda] \rangle) = \begin{cases} \min\{s + 1, 4\lambda\} & \text{if } \pi(U) = \pi(V) \vee \min\{\pi(U), \pi(V)\} \geq e, \\ 0 & \text{if } \pi(U) \neq \pi(V) \wedge \min\{\pi(U), \pi(V)\} = \$, \\ \text{select}_{\mathbb{X}}(\langle U \rangle, 2) - 1 & \text{if } \$ \neq \pi(U) < \min\{\pi(V), e\}, \\ \text{select}_{\mathbb{X}}(\langle V \rangle, 2) - 1 & \text{otherwise.} \end{cases}$$

► **Lemma 6.4.** *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma_{\mathbb{S}}^+$, $n = |T_1 \cdots T_d|$, $n' = \max\{|T_k| \mid k \in [1..d]\}$, $i, j \in [1..n]$, and $i < j$. Then $\text{lcp}(\langle C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i])^{\omega}[\dots 4n'] \rangle, \langle C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[j])^{\omega}[\dots 4n'] \rangle) = \min\{\text{LCP}_{\mathcal{T}}^{\circ}[k] \mid k \in [i + 1..j]\} = \text{RNV}_{\text{LCP}_{\mathcal{T}}^{\circ}}(i + 1, j, -1)$.*

► **Lemma 6.5** ([25]). *A dynamic string of length n over $[0..\sigma]$ with $\sigma \leq n^{O(1)}$ can be stored in a data structure occupying $O(n \lg \sigma)$ bits of space, supporting insertion, deletion and the queries access, rank and select in $O(\frac{\lg n}{\lg \lg n})$ time.*

Leveraging the techniques presented in Section 4.1, Lemma 6.3, Lemma 6.4 and Lemma 6.5, we arrive at the following.

► **Corollary 6.6.** *Let $R \in \Sigma_{\mathbb{S}}^+$, $\rho = |R|$, $R[\rho] = \$$, and $\text{rank}_{\mathbb{S}}(R, \rho) = 1$. Then $\text{LCP}_{\mathcal{R}}^{\circ}$ can be constructed alongside the ceBWT index of $\mathcal{R}$ in $O(\rho \lg \rho)$ bits of space and $O(\rho \frac{\lg \sigma \lg \rho}{\lg \lg \rho})$ time.*

We adapt the techniques for the computation of $\text{LCP}_{\mathcal{T}}^{\circ}$ presented in Section 4.2 to compute $\text{LCP}_{\mathcal{T}}^{\circ}$ alongside the ceBWT index of $\mathcal{T}$. For the $(k - 1)$ -th extension with $k \in [1..d]$, we obtain $\text{LCP}_{\{T_k^{\circ}\}}^{\circ}$ with Corollary 6.6 and the adapted helper values by computing the circular matching statistics between $\{T_1, \dots, T_{k-1}\}$ and T_k . We give a summary.

► **Lemma 6.7.** *Let $\emptyset \neq \mathcal{T} = \{T_1, \dots, T_d\} \subset \Sigma^+$ and $n = |T_1 \cdots T_d|$. Then $\text{LCP}_{\mathcal{T}}^{\circ}$ can be constructed alongside the ceBWT index of $\mathcal{T}$ in $O(n \lg n)$ bits of space and $O(n \frac{\lg^2 n}{\lg \lg n})$ time.*

7 Conclusion

We presented a novel index data structure, called the ceBWT index. The ceBWT index is based on Kim and Cho's index for Cartesian tree matching. Contrary to its predecessor, the ceBWT index supports multiple input texts and circular pattern matching by inheriting techniques from the extended Burrows–Wheeler transform. The ceBWT index can be constructed in $O(n \frac{\lg \sigma \lg n}{\lg \lg n})$ time while using $O(n \lg \sigma)$ bits of working space. In the dynamic setting, it supports incremental indexing, making it adaptable for dynamically growing datasets. The dynamic ceBWT index can answer COUNT in $O(m \frac{\lg \sigma \lg n}{\lg \lg n})$ time for a pattern of length m . For the static setting, we can compactify our index to $3n + o(n)$ bits of space, supporting then COUNT in $O(m)$ time. As a possible application, we showed how to compute the circular matching statistics between a set of texts and an m -length pattern in $O(m \lg n)$ time. Further applications of interest are melody matching and financial pattern detection, where our index can be applied to detect patterns in time series data.

References

- 1 Bastien Auvray, Julien David, Richard Groult, and Thierry Lecroq. Approximate cartesian tree matching: An approach using swaps. In *Proc. SPIRE*, volume 14240 of *LNCS*, pages 49–61, 2023.
- 2 Christina Boucher, Davide Cenzato, Zsuzsanna Lipták, Massimiliano Rossi, and Marinella Sciortino. r-indexing the eBWT. In *Proc. SPIRE*, volume 12944 of *LNCS*, pages 3–12, 2021.
- 3 William I. Chang and Eugene L. Lawler. Sublinear approximate string matching and biological applications. *Algorithmica*, 12(4/5):327–344, 1994.
- 4 Maxime Crochemore and Wojciech Rytter. *Jewels of Stringology: Text Algorithms*. World Scientific, 2002.
- 5 Erik D. Demaine, Gad M. Landau, and Oren Weimann. On Cartesian trees and range minimum queries. *Algorithmica*, 68(3):610–625, 2014.
- 6 Simone Faro, Thierry Lecroq, Kunsoo Park, and Stefano Scafiti. On the longest common Cartesian substring problem. *The Computer Journal*, 66(4):907–923, 2022.
- 7 Paolo Ferragina and Giovanni Manzini. Opportunistic data structures with applications. In *Proc. FOCS*, pages 390–398, 2000.
- 8 Nathan J. Fine and Herbert S. Wilf. Uniqueness theorems for periodic functions. *Proceedings of the American Mathematical Society*, 16(1):109–114, 1965.
- 9 Johannes Fischer. Optimal succinctness for range minimum queries. In *Proc. LATIN*, volume 6034 of *LNCS*, pages 158–169, 2010.
- 10 Peter Foster, Anssi Klapuri, and Simon Dixon. A method for identifying repetition structure in musical audio based on time series prediction. In *Proc. EUSIPCO*, pages 1299–1303. IEEE, 2012.
- 11 Tak-Chung Fu, Korris Fu-Lai Chung, Robert Wing Pong Luk, and Chak-man Ng. Stock time series pattern matching: Template-based vs. rule-based approaches. *Eng. Appl. Artif. Intell.*, 20(3):347–364, 2007.
- 12 Mitsuru Funakoshi, Takuya Mieno, Yuto Nakashima, Shunsuke Inenaga, Hideo Bannai, and Masayuki Takeda. Computing maximal palindromes in non-standard matching models. In *Proc. IWOCA*, volume 14764 of *LNCS*, pages 165–179, 2024.
- 13 Paweł Gawrychowski, Samah Ghazawi, and Gad M. Landau. On indeterminate strings matching. In *Proc. CPM*, volume 161 of *LIPIcs*, pages 14:1–14:14, 2020.
- 14 Daiki Hashimoto, Diptarama Hendrian, Dominik Köppl, Ryo Yoshinaka, and Ayumi Shinohara. Computing the parameterized Burrows–Wheeler transform online. In *Proc. SPIRE*, volume 13617 of *LNCS*, pages 70–85, 2022.
- 15 Yuzuru Hiraga. Structural recognition of music by pattern matching. In *Proc. ICMC*. Michigan Publishing, 1997.
- 16 Wing-Kai Hon, Tsung-Han Ku, Chen-Hua Lu, Rahul Shah, and Sharma V. Thankachan. Efficient algorithm for circular Burrows–Wheeler transform. In *Proc. CPM*, volume 7354 of *LNCS*, pages 257–268, 2012.
- 17 Kento Iseri, Tomohiro I, Diptarama Hendrian, Dominik Köppl, Ryo Yoshinaka, and Ayumi Shinohara. Breaking a barrier in constructing compact indexes for parameterized pattern matching. In *Proc. ICALP*, volume 297 of *LIPIcs*, pages 89:1–89:19, 2024.
- 18 Natsumi Kikuchi, Diptarama Hendrian, Ryo Yoshinaka, and Ayumi Shinohara. Computing covers under substring consistent equivalence relations. In *Proc. SPIRE*, volume 12303 of *LNCS*, pages 131–146, 2020.
- 19 Jinil Kim, Peter Eades, Rudolf Fleischer, Seok-Hee Hong, Costas S. Iliopoulos, Kunsoo Park, Simon J. Puglisi, and Takeshi Tokuyama. Order-preserving matching. *Theor. Comput. Sci.*, 525:68–79, 2014.
- 20 Sung-Hwan Kim and Hwan-Gue Cho. A compact index for Cartesian tree matching. In *Proc. CPM*, volume 191 of *LIPIcs*, pages 18:1–18:19, 2021.
- 21 Sungmin Kim and Yo-Sub Han. Approximate Cartesian tree pattern matching. In *Proc. DLT*, volume 14791 of *LNCS*, pages 189–202, 2024.

- 22 Marcin Kubica, Tomasz Kulczyński, Jakub Radoszewski, Wojciech Rytter, and Tomasz Waleń. A linear time algorithm for consecutive permutation pattern matching. *Inf. Process. Lett.*, 113(12):430–433, 2013.
- 23 Sabrina Mantaci, Antonio Restivo, Giovanna Rosone, and Marinella Sciortino. An extension of the Burrows–Wheeler transform. *Theor. Comput. Sci.*, 387(3):298–312, 2007.
- 24 Yoshiaki Matsuoka, Takahiro Aoki, Shunsuke Inenaga, Hideo Bannai, and Masayuki Takeda. Generalized pattern matching and periodicity under substring consistent equivalence relations. *Theor. Comput. Sci.*, 656:225–233, 2016.
- 25 J. Ian Munro, Yakov Nekrich, and Jeffrey Scott Vitter. Dynamic data structures for document collections and graphs. In *Proc. PODS*, pages 277–289, 2015.
- 26 Gonzalo Navarro and Kunihiro Sadakane. Fully functional static and dynamic succinct trees. *ACM Trans. Algorithms*, 10(3):16:1–16:39, 2014.
- 27 Akio Nishimoto, Noriki Fujisato, Yuto Nakashima, and Shunsuke Inenaga. Position heaps for Cartesian-tree matching on strings and tries. In *Proc. SPIRE*, volume 12944 of *LNCS*, pages 241–254, 2021.
- 28 Enno Ohlebusch, Simon Gog, and Adrian Kügel. Computing matching statistics and maximal exact matches on compressed full-text indexes. In *Proc. SPIRE*, volume 6393 of *LNCS*, pages 347–358, 2010.
- 29 Tsubasa Oizumi, Takeshi Kai, Takuya Mieno, Shunsuke Inenaga, and Hiroki Arimura. Cartesian tree subsequence matching. In *Proc. CPM*, volume 223 of *LIPIcs*, pages 14:1–14:18, 2022.
- 30 Eric M. Osterkamp and Dominik Köppl. Extending the parameterized Burrows–Wheeler transform. In *Proc. DCC*, pages 143–152, 2024.
- 31 Eric M. Osterkamp and Dominik Köppl. Extending the Burrows–Wheeler transform for Cartesian tree matching and constructing it. In *Proc. CPM*, volume 331 of *LIPIcs*, pages 26:1–26:17, 7 2025.
- 32 Sung Gwan Park, Magsarjav Bataa, Amihood Amir, Gad M. Landau, and Kunsoo Park. Finding patterns and periods in Cartesian tree matching. *Theor. Comput. Sci.*, 845:181–197, 2020.
- 33 Siwoo Song, Geonmo Gu, Cheol Ryu, Simone Faro, Thierry Lecroq, and Kunsoo Park. Fast algorithms for single and multiple pattern Cartesian tree matching. *Theor. Comput. Sci.*, 849:47–63, 2021.
- 34 Jean Vuillemin. A unifying look at data structures. *Commun. ACM*, 23(4):229–239, 1980.

A

 Properties of the Rotational Cartesian Tree Signature Encoding

As a result of independent interest, we further examine the rotational Cartesian tree signature encoding and how it relates to the rotational parent distance encoding. We first give another result for the latter.

► **Lemma A.1.** *Let $V, U \in \Sigma^+$. Then $\langle V \rangle_r = \langle U \rangle_r$ if and only if $V \approx U$.*

Proof. ($\Rightarrow$) Let $U \not\approx V$. If $|U| \neq |V|$, then $\langle U \rangle_r \neq \langle V \rangle_r$. Hence, assume $|U| = |V|$. Then $\langle U \rangle \neq \langle V \rangle$ by Lemma 2.2. Let $i \in [1..|U|]$ minimal such that $\langle U \rangle[i] \neq \langle V \rangle[i]$. If $\max\{\langle U \rangle[i], \langle V \rangle[i]\} < \infty$, then $\langle U \rangle_r[i] = \langle U \rangle[i] \neq \langle V \rangle[i] = \langle V \rangle_r[i]$, i.e., $\langle V \rangle_r \neq \langle U \rangle_r$. Thus, assume $\max\{\langle U \rangle[i], \langle V \rangle[i]\} = \infty$. Without loss of generality, $\langle U \rangle[i] = \infty$. Then $\langle V \rangle_r[i] = \langle V \rangle[i] < i \leq \langle U \rangle_r[i]$, i.e., $\langle V \rangle_r \neq \langle U \rangle_r$.
 ($\Leftarrow$) Let $U \approx V$. Then $\langle U \rangle = \langle V \rangle$ by Lemma 2.2. In particular, $|\langle U \rangle_r| = |\langle V \rangle_r|$. Assume $\langle U \rangle_r \neq \langle V \rangle_r$. Let $i \in [1..|U|]$ minimal such that $\langle U \rangle_r \neq \langle V \rangle_r$. Since $\langle W \rangle[k] < \infty$ implies $\langle W \rangle[k] = \langle W \rangle_r[k]$ for each $W \in \Sigma^+$ and $k \in [1..|W|]$, $\langle U \rangle[i] = \langle V \rangle[i] = \infty$, i.e., $\min\{\langle V \rangle_r[i], \langle U \rangle_r[i]\} \geq i$. Without loss of generality, $\langle V \rangle_r[i] < \langle U \rangle_r[i] \leq |V|$. Let $j = i - \langle V \rangle_r[i] + |V|$. Then $\langle V \rangle[j] < \infty$, i.e., $V[j]$ occurs at least twice in V . However, $U[j] \neq U[i]$, which contradicts $U \approx V$ since $V[i] = V[j]$. Hence, $\langle U \rangle_r = \langle V \rangle_r$. ◀

The following result shows that the rotational Cartesian tree signature encoding is commutative with rotation.

► **Lemma A.2.** *For every $V \in \Sigma_{\mathcal{S}}^+$ and $i, j \in [1..|V|]$, $\llbracket \text{Rot}(V, i) \rrbracket[j] = \text{Rot}(\llbracket V \rrbracket, i)[j]$.*

Proof. Fix some $V \in \Sigma_{\mathcal{S}}^+$ with $\lambda = |V|$ and $i, j \in [1..\lambda]$. It is easy to see that the statement holds if $\text{Rot}(V, i)[j] = \$$. Thus, assume $\text{Rot}(V, i)[j] \neq \$$. If $i + j \bmod \lambda \neq 0$, then

$$\begin{aligned} \llbracket \text{Rot}(V, i) \rrbracket[j] &= \text{rank}_x(\langle \text{Rot}(\text{Rot}(V, i), j) \rangle, \lambda) \\ &\quad - \text{rank}_x(\langle \text{Rot}(V, i)[j] \cdot \text{Rot}(\text{Rot}(V, i), j) \rangle[2..], \lambda) \\ &= \text{rank}_x(\langle \text{Rot}(V, i + j) \rangle, \lambda) \\ &\quad - \text{rank}_x(\langle V[i + j \bmod \lambda] \cdot \text{Rot}(V, i + j) \rangle[2..], \lambda) \\ &= \llbracket V \rrbracket[i + j \bmod \lambda] \\ &= \text{Rot}(\llbracket V \rrbracket, i)[j]. \end{aligned}$$

If $i + j \bmod \lambda = 0$, then the statement follows similarly with $\text{Rot}(\llbracket V \rrbracket, i)[j] = \llbracket V \rrbracket[\lambda]$ and $\text{Rot}(V, i)[j] = V[\lambda]$. ◀

► **Corollary A.3.** *Let $V, U \in \Sigma_{\mathcal{S}}^+$. Then $\llbracket V \rrbracket = \llbracket U \rrbracket$ if and only if $V \approx U$.*

Then Lemma 2.2 implies that the rotational modified parent distance encoding can be substituted for the rotational tree signature encoding in the definition of the ω -preorder and the permutation $\text{prev}_{\mathcal{T}}$. The following statement is another consequence of Lemma A.2.

► **Corollary A.4.** *Let $\emptyset \neq \mathcal{T} \subset \Sigma_{\mathcal{S}}^+$ and n the accumulated length of all texts in $\mathcal{T}$. Then $F_{\mathcal{T}}[i] = \pi(C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i]))$ and $L_{\mathcal{T}}[i] = \llbracket C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i]) \rrbracket \llbracket C_{\mathcal{T}}(\text{CA}_{\mathcal{T}}[i]) \rrbracket$ for each $i \in [1..n]$.*

Hence, we can define both $F_{\mathcal{T}}$ and $L_{\mathcal{T}}$ from the conjugate array and the $\llbracket \cdot \rrbracket$ -encoded conjugates considered in $\mathcal{T}$, and then define the LF-mapping with the statement of Corollary 3.10.

B

 Variables

The table below gives an overview of variables frequently used throughout the article.

<i>variable name</i>	<i>description</i>
$\Sigma, \Sigma_{\$}$	alphabet
$b, \$, \infty$	symbol ($\forall b \in \Sigma \cup \{0\} : \$ < b < \infty$)
σ	alphabet size
$J_0, J_1, \dots$	intervals
$\ell, \ell', \ell'', \ell_0, \ell_1, \dots$	left interval boundary
$r, r', r'', r_0, r_1, \dots$	right interval boundary
U, V, W, X, Y	string
$\text{root}(V), \text{exp}(V)$	the root and exponent of V
$ V $	length of V
$\text{lcp}(U, W)$	length of the longest common prefix of U and W
ε	empty string
$\langle V \rangle$	parent distance encoding of V
$\text{lcp}^{\infty}(U, W)$	rank of ∞ in longest common prefix of $\langle U \rangle$ and $\langle W \rangle$
$\langle V \rangle_r$	rotational parent distance encoding of V
$\llbracket V \rrbracket$	rotational Cartesian tree signature encoding of V
$\pi(V)$	first symbol of $\llbracket V \rrbracket$
$c, d, e, h, i, j, k, p, q, v, x, y, z, \lambda, \rho$	integer value
$\text{Rot}(V, i)$	the i -th left rotation of V
$Y, E, E_1, E_2, \dots$	auxiliary bit string
$R, S, T, T_1, T_2, \dots$	text string
$n, n_1, n_2, \dots$	text length of $T, T_1, T_2, \dots$
$P, P_1, P_2, \dots$	pattern string
m	pattern length
$\mathcal{R}, \mathcal{S}, \mathcal{T}$	set of text strings
$C_{\mathcal{T}}(i)$	conjugate of a text in $\mathcal{T}$
$\text{CA}_{\mathcal{T}}, \text{CA}_{\mathcal{T}}^{-1}$	conjugate array of $\mathcal{T}$ and its inverse
$\text{CR}_{\mathcal{T}}(P)$	conjugate range of P in $\mathcal{T}$
$\text{LF}_{\mathcal{T}}, \text{FL}_{\mathcal{T}}$	LF-/FL-mapping of $\mathcal{T}$
$\text{F}_{\mathcal{T}}, \text{L}_{\mathcal{T}}, \text{LCP}_{\mathcal{T}}^{\infty}$	ceBWT index of $\mathcal{T}$
$\text{cnt}_{\mathcal{T}}(\text{Rot}(S, i))$	helper value for the extension of $\text{F}_{\mathcal{T}}, \text{L}_{\mathcal{T}}$ and $\text{LCP}_{\mathcal{T}}^{\infty}$ to $\text{F}_{\mathcal{T} \cup \{S\}}, \text{L}_{\mathcal{T} \cup \{S\}}$ and $\text{LCP}_{\mathcal{T} \cup \{S\}}^{\infty}$, respectively
$\text{plcp}_{\mathcal{T}}^{\infty}(\text{Rot}(S, i)), \text{slcp}_{\mathcal{T}}^{\infty}(\text{Rot}(S, i))$	helper value for extending $\text{LCP}_{\mathcal{T}}^{\infty}$ to $\text{LCP}_{\mathcal{T} \cup \{S\}}^{\infty}$